Introduction to Computer Architecture and Systems

Chapter 6
From LC-3 to Assembly

Fengkai Wang(Iori Oikawa), Fall 2025 @ Drew University

Previously of CSCI 260…

- Essential Improvements / New Concepts
  - New Format: Label - Opcode - Operand - Comment
  - Labels
  - Comments
  - Pseudo-Ops: .ORIG .BLKW .Fill .END and .STRINGZ
- Machine Code to Assembly

Assembly

And Machine code

Address	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
x3000	0	1	0	1	0	1	0	0	1	0	1	0	0	0	0	0	R2 <- 0
x3001	0	0	1	0	0	1	1	0	0	0	0	1	0	0	0	0	R3 <- M[x3012]
x3002	1	1	1	1	0	0	0	0	0	0	1	0	0	0	1	1	TRAP x23
x3003	0	1	1	0	0	0	1	0	1	1	0	0	0	0	0	0	R1 <- M[R3]
x3004	0	0	0	1	1	0	0	0	0	1	1	1	1	1	0	0	R4 <- R1-4
x3005	0	0	0	0	0	1	0	0	0	0	0	0	1	0	0	0	BRz x300E
x3006	1	0	0	1	0	0	1	0	0	1	1	1	1	1	1	1	R1 <- NOT R1
x3007	0	0	0	1	0	0	1	0	0	1	1	0	0	0	0	1	R1 <- R1 + 1
x3008	0	0	0	1	0	0	1	0	0	1	0	0	0	0	0	0	R1 <- R1 + R0
x3009	0	0	0	0	1	0	1	0	0	0	0	0	0	0	0	1	BRnp x300B
x300A	0	0	0	1	0	1	0	0	1	0	1	0	0	0	0	1	R2 <- R2 + 1
x300B	0	0	0	1	0	1	1	0	1	1	1	0	0	0	0	1	R3 <- R3 + 1
x300C	0	1	1	0	0	0	1	0	1	1	0	0	0	0	0	0	R1 <- M[R3]
x300D	0	0	0	0	1	1	1	1	1	1	1	1	0	1	1	0	BRnzp x3004
x300E	0	0	1	0	0	0	0	0	0	0	0	0	0	1	0	0	R0 <- M[x3013]
x300F	0	0	0	1	0	0	0	0	0	0	0	0	0	0	1	0	R0 <- R0 + R2
x3010	1	1	1	1	0	0	0	0	0	0	1	0	0	0	0	1	TRAP x21
x3011	1	1	1	1	0	0	0	0	0	0	1	0	0	1	0	1	TRAP x25
x3012	Starting address of file																
x3013	0	0	0	0	0	0	0	0	0	0	1	1	0	0	0	0	ASCII TEMPLATE

```
01   ;
02   ; Program to count occurrences of a character in a file.
03   ; Character to be input from the keyboard.
04   ; Result to be displayed on the monitor.
05   ; Program works only if no more than 9 occurrences are found.
06   ;
07   ;
08   ; Initialization
09   ;
0A            .ORIG   x3000
0B            AND     R2,R2,#0        ; R2 is counter, initialize to 0
0C            LD      R3,PTR          ; R3 is pointer to characters
0D            TRAP    x23             ; R0 gets character input
0E            LDR     R1,R3,#0        ; R1 gets the next character
0F   ;
10   ; Test character for end of file
11   ;
12   ;
13   TEST     ADD     R4,R1,#-4       ; Test for EOT
14            BRz     OUTPUT          ; If done, prepare the output
15   ;
16   ; Test character for match.  If a match, increment count.
17   ;
18            NOT     R1,R1
19            ADD     R1,R1,#1        ; R1 <-- -R1
1A            ADD     R1,R1,R0        ; R1 <-- R0-R1. If R1=0, a match!
1B            BRnp    GETCHAR         ; If no match, do not increment
1C            ADD     R2,R2,#1
1D   ;
1E   ; Get next character from the file
1F   ;
20   GETCHAR ADD     R3,R3,#1        ; Increment the pointer
21            LDR     R1,R3,#0        ; R1 gets the next character to test
22            BRnzp   TEST
23   ;
24   ; Output the count.
25   ;
26   OUTPUT   LD      R0,ASCII        ; Load the ASCII template
27            ADD     R0,R0,R2        ; Convert binary to ASCII
28            TRAP    x21             ; ASCII code in R0 is displayed
29            TRAP    x25             ; Halt machine
2A   ;
2B   ; Storage for pointer and ASCII template
2C   ;
2D   ASCII    .FILL   x0030
2E   PTR      .FILL   x4000
2F            .END
```

Wait…

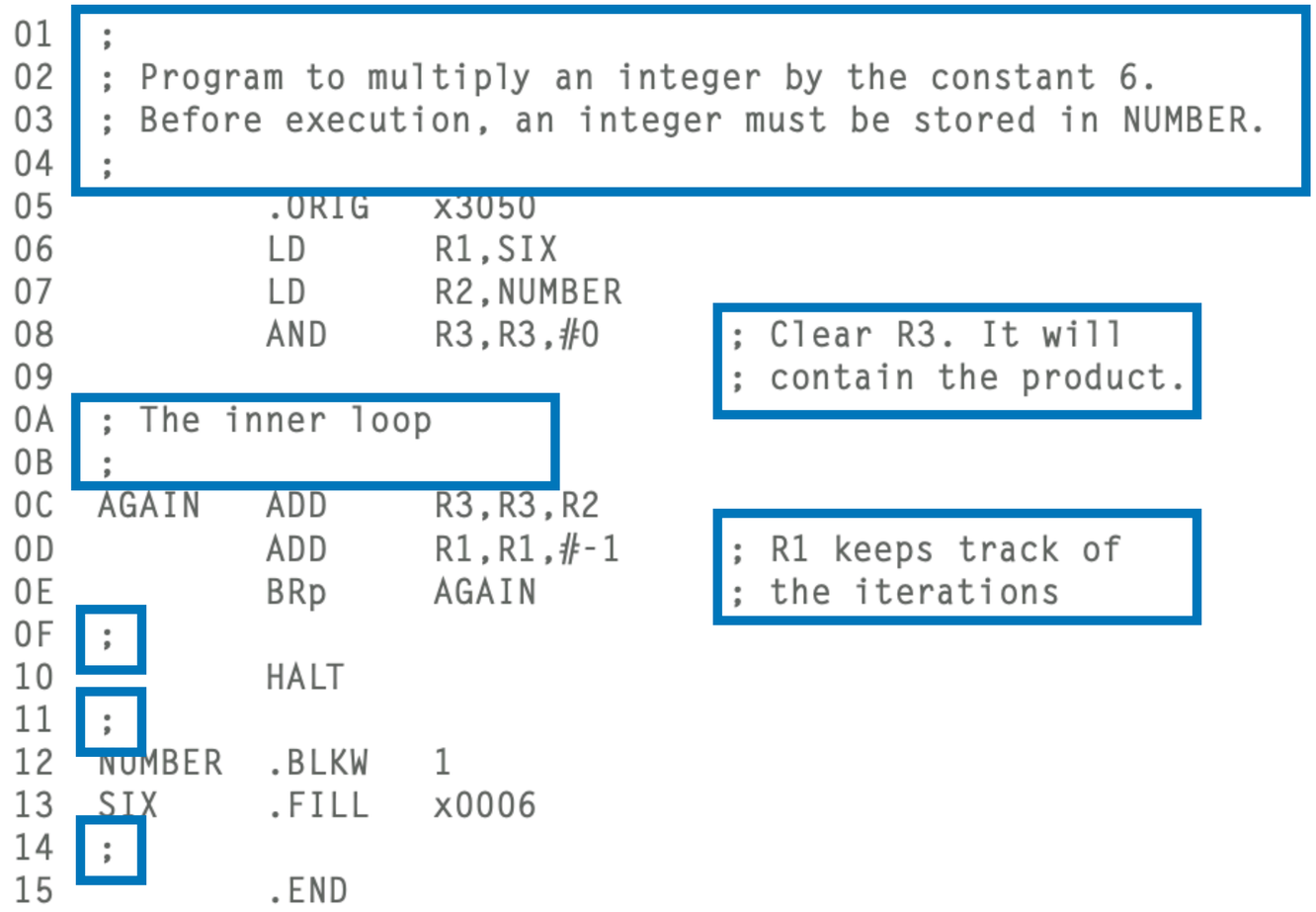
In order to have labels,
comments, and pseudo-ops,
What do we need to introduce?

Example: Multiply a number by 6

```
01  ;
02  ; Program to multiply an integer by the constant 6.
03  ; Before execution, an integer must be stored in NUMBER.
04  ;
05          .ORIG   x3050
06          LD      R1,SIX
07          LD      R2,NUMBER
08          AND     R3,R3,#0        ; Clear R3. It will
09                                  ; contain the product.
0A  ; The inner loop
0B  ;
0C  AGAIN   ADD     R3,R3,R2
0D          ADD     R1,R1,#-1       ; R1 keeps track of
0E          BRp     AGAIN           ; the iterations
0F  ;
10          HALT
11  ;
12  NUMBER  .BLKW   1
13  SIX     .FILL   x0006
14  ;
15          .END
```

Add-ons: Comments

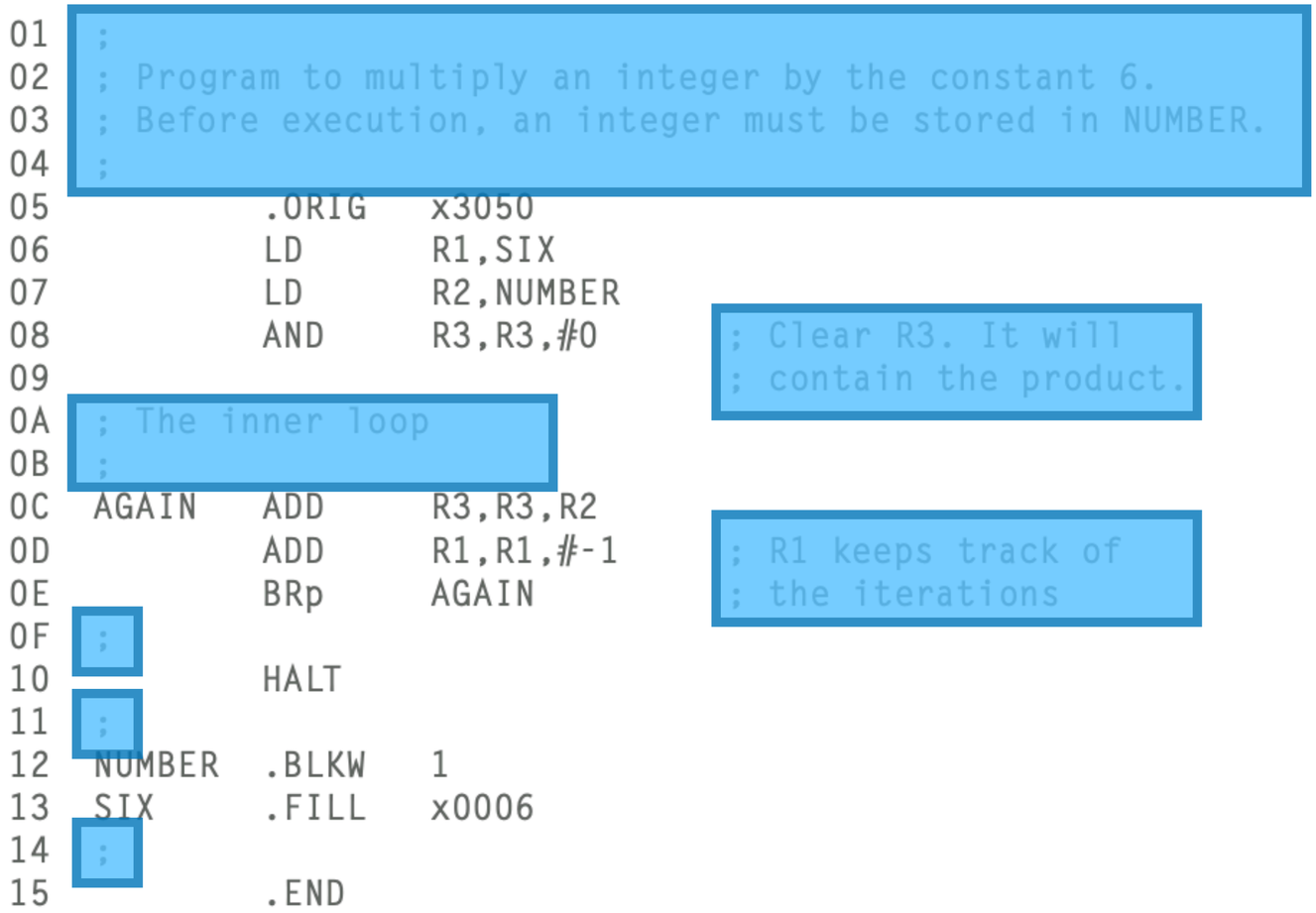
Information starting with ‘;’



```
01  ;
02  ; Program to multiply an integer by the constant 6.
03  ; Before execution, an integer must be stored in NUMBER.
04  ;
05          .ORIG   x3050
06          LD      R1,SIX
07          LD      R2,NUMBER
08          AND     R3,R3,#0        ; Clear R3. It will
09                                  ; contain the product.
0A  ; The inner loop
0B  ;
0C  AGAIN   ADD     R3,R3,R2
0D          ADD     R1,R1,#-1       ; R1 keeps track of
0E          BRp     AGAIN           ; the iterations
0F  ;
10          HALT
11  ;
12  NUMBER  .BLKW   1
13  SIX     .FILL   x0006
14  ;
15          .END
```

Treatment: Comments

In order to use comments, we need to get rid of them at run time.



```
01  ;
02  ; Program to multiply an integer by the constant 6.
03  ; Before execution, an integer must be stored in NUMBER.
04  ;
05          .ORIG   x3050
06          LD      R1,SIX
07          LD      R2,NUMBER
08          AND     R3,R3,#0        ; Clear R3. It will
09                                  ; contain the product.
0A  ; The inner loop
0B  ;
0C  AGAIN   ADD     R3,R3,R2
0D          ADD     R1,R1,#-1       ; R1 keeps track of
0E          BRp     AGAIN           ; the iterations
0F  ;
10          HALT
11  ;
12  NUMBER  .BLKW   1
13  SIX     .FILL   x0006
14  ;
15          .END
```

- This means comments, as useless to machine, must be discarded before translating the assembly into machine code.
- Opens the idea to ‘preprocessing’.

Add-ons: Labels

Instead of counting number of lines, we can use words now

```
01  ;
02  ; Program to multiply an integer by the constant 6.
03  ; Before execution, an integer must be stored in NUMBER.
04  ;
05          .ORIG   x3050
06          LD      R1,SIX
07          LD      R2,NUMBER
08          AND     R3,R3,#0        ; Clear R3. It will
09                                  ; contain the product.
0A  ; The inner loop
0B  ;
0C  AGAIN   ADD     R3,R3,R2
0D          ADD     R1,R1,#-1       ; R1 keeps track of
0E          BRp     AGAIN           ; the iterations
0F  ;
10          HALT
11  ;
12  NUMBER  .BLKW   1
13  SIX     .FILL   x0006
14  ;
15          .END
```

Treatment: Labels

In order to use user-defined labels, we need a table to build relationships…

```
01  ;
02  ; Program to multiply an integer by the constant 6.
03  ; Before execution, an integer must be stored in NUMBER.
04  ;
05          .ORIG   x3050
06          LD      R1,SIX
07          LD      R2,NUMBER
08          AND     R3,R3,#0        ; Clear R3. It will
09                                  ; contain the product.
0A  ; The inner loop
0B  ;
0C  AGAIN   ADD     R3,R3,R2
0D          ADD     R1,R1,#-1       ; R1 keeps track of
0E          BRp     AGAIN           ; the iterations
0F  ;
10          HALT
11  ;
12  NUMBER  .BLKW   1
13  SIX     .FILL   x0006
14  ;
15          .END
```

Symbol	Address
SIX	x3059
NUMBER	x3058
AGAIN	X3054

Add-ons: Pseudo-Ops

All words begin with '.'

```
01  ;
02  ; Program to multiply an integer by the constant 6.
03  ; Before execution, an integer must be stored in NUMBER.
04  ;
05          .ORIG   x3050
06          LD      R1,SIX
07          LD      R2,NUMBER
08          AND     R3,R3,#0        ; Clear R3. It will
09                                  ; contain the product.
0A  ; The inner loop
0B  ;
0C  AGAIN   ADD     R3,R3,R2
0D          ADD     R1,R1,#-1       ; R1 keeps track of
0E          BRp     AGAIN           ; the iterations
0F  ;
10          HALT
11  ;
12  NUMBER  .BLKW   1
13  SIX     .FILL   x0006
14  ;
15          .END
```

Treatment: Pseudo-Ops

All words begin with '.'

```
01  ;
02  ; Program to multiply an integer by the constant 6.
03  ; Before execution, an integer must be stored in NUMBER.
04  ;
05          .ORIG   x3050
06          LD      R1,SIX
07          LD      R2,NUMBER
08          AND     R3,R3,#0        ; Clear R3. It will
09                                  ; contain the product.
0A  ; The inner loop
0B  ;
0C  AGAIN   ADD     R3,R3,R2
0D          ADD     R1,R1,#-1       ; R1 keeps track of
0E          BRp     AGAIN           ; the iterations
0F  ;
10          HALT
11  ;
12  NUMBER  .BLKW   1
13  SIX     .FILL   x0006
14  ;
15          .END
```

Translating Pseudo-Ops to Machine Code

These actions are now performed by the assembler.

- .ORIG = Set the Program Counter's Initial(ORIGINAL) Position

```
.ORIG   x3050
```

- .BLKW = set aside the next location in the program and initialize it with the value of the operand. The value can be either a number or a label.

```
.BLKW   1
```

- .FILL = set aside some number of sequential memory locations (i.e., a BLocK of Words) in the program. The actual number is the operand of the .BLKW pseudo-op.

```
.FILL   x0006
```

- .END = tells the assembler it has reached the end of the program and need not even look at anything after it. That is, any characters that come after .END will not be processed by the assembler. But .END itself does NOT stop the program.

Translating Pseudo-Ops to Machine Code

- .STRINGZ tells the assembler to initialize a sequence of n + 1 memory locations. The argument is a sequence of n characters inside **double quotation marks**. The first n words of memory are initialized with the zero-extended ASCII codes of the corresponding characters in the string. The final word of memory is initialized to 0. The last word, containing x0000, provides a convenient **sentinel** for processing the string of ASCII codes.

```
        .ORIG     x3010
HELLO   .STRINGZ  "Hello, World!"
```



```
x3010: x0048
x3011: x0065
x3012: x006C
x3013: x006C
x3014: x006F
x3015: x002C
x3016: x0020
x3017: x0057
x3018: x006F
x3019: x0072
x301A: x006C
x301B: x0064
x301C: x0021
x301D: x0000
```

Summary: LC-3's Two-Pass Assembly Process

(Preprocess), Creating the Symbol Table, then Generating Machine Code

- Preprocess: Identify the comments and delete them (ignoring all characters between ';' and the end of the line).
- Creating Symbol Table:
  - Use Location Counter(LC) to track each instruction's assigned location, LC is initially set by .ORIG
  - Assembler examine each instruction increasing LC. Once find a label, it records the label and LC's location, until reaching .END
- Create Machine Code: (Continue)

Summary: LC-3's Two-Pass Assembly Process

(Preprocess), Creating the Symbol Table, then Generating Machine Code

- Preprocess: Identify the comments and delete them (ignoring all characters between ';' and the end of the line).
- First Pass - Creating Symbol Table:
  - Use Location Counter(LC) to track each instruction's assigned location, LC is initially set by .ORIG
  - Assembler examine each instruction increasing LC. Once find a label, it records the label and LC's location, until reaching .END
- Second Pass - Create Machine Code: (Continue)

Example

How does the assembly process work on this code?

```
            .ORIG     x3005
             LEA      R2, DATA
             LDR      R4, R2, #0
LOOP         ADD      R4, R4, #-3
             BRzp     LOOP
             TRAP     x25
DATA        .FILL      x000B
            .END
```

Example

How does the assembly process work on this code?

```
            .ORIG      x3005
             LEA       R2, DATA
             LDR       R4, R2, #0
LOOP         ADD       R4, R4, #-3
             BRzp      LOOP
             TRAP      x25
DATA        .FILL       x000B
            .END
```

Preprocess Build Symbol Table Generate Machine Code

Example

How does the assembly process work on this code?

```
        .ORIG     x3005
         LEA      R2, DATA
         LDR      R4, R2, #0
LOOP     ADD      R4, R4, #-3
         BRzp     LOOP
         TRAP     x25
DATA     .FILL     x000B
         .END
```

Preprocess: Getting Rid of Comments

Since there's no comments

We are done with preprocessing.

Preprocess Build Symbol Table Generate Machine Code

Example

How does the assembly process work on this code?

LC -> x3005

```
        .ORIG    x3005
        LEA      R2, DATA
        LDR      R4, R2, #0
LOOP    ADD      R4, R4, #-3
        BRzp     LOOP
        TRAP     x25
DATA    .FILL    x000B
        .END
```

Build Symbol Table:
We set LC to x3005
due to .ORIG

Preprocess Build Symbol Table Generate Machine Code

Example

How does the assembly process work on this code?

```
                        .ORIG     x3005
      x3005              LEA      R2, DATA
      x3006              LDR      R4, R2, #0
LC -> x3007 LOOP         ADD      R4, R4, #-3
                         BRzp     LOOP
                         TRAP     x25
            DATA        .FILL     x000B
                        .END
```

Build Symbol Table: Moving LC until we find the first label ‘LOOP’

Record the label and LC’s position.

Symbol	Address
LOOP	x3007

Preprocess Build Symbol Table Generate Machine Code

Example

How does the assembly process work on this code?

```
                    .ORIG    x3005
    x3005            LEA     R2, DATA
    x3006            LDR     R4, R2, #0
    x3007 LOOP       ADD     R4, R4, #-3
    x3008            BRzp    LOOP
    x3009            TRAP    x25
LC -> x300A DATA    .FILL    x000B
                    .END
```

Build Symbol Table: Continue moving LC and we find the second label ‘LOOP’

Record the label and LC’s position.

Symbol	Address
LOOP	x3008
DATA	x300B

Preprocess Build Symbol Table Generate Machine Code

Example

How does the assembly process work on this code?

```
                    .ORIG    x3005
    x3005            LEA     R2, DATA
    x3006            LDR     R4, R2, #0
    x3007 LOOP       ADD     R4, R4, #-3
    x3008            BRzp    LOOP
    x3009            TRAP    x25
LC -> x300A DATA    .FILL    x000B
                    .END
```

Build Symbol Table:
LC reached to .END

Symbol Table Complete

Symbol	Address
LOOP	x3008
DATA	x300B

Preprocess

Build Symbol Table

Generate Machine Code

Example

How does the assembly process work on this code?

```
                        .ORIG     x3005
LC -> x3005              LEA      R2, DATA
      x3006              LDR      R4, R2, #0
      x3007 LOOP         ADD      R4, R4, #-3
      x3008              BRzp     LOOP
      x3009              TRAP     x25
      x300A DATA        .FILL      x000B
                        .END
```

Generate Machine Code:
Reset LC to x3005

Preprocess

Build Symbol Table

Generate Machine Code

Example

How does the assembly process work on this code?

```
                        .ORIG    x3005
LC -> x3005              LEA     R2, DATA
      x3006              LDR     R4, R2, #0
      x3007 LOOP         ADD     R4, R4, #-3
      x3008              BRzp    LOOP
      x3009              TRAP    x25
      x300A DATA        .FILL     x000B
                        .END
```

Generate Machine Code: Write each line of instruction replacing labels into corresponding addresses.

Preprocess

Build Symbol Table

Generate Machine Code

Example

How does the assembly process work on this code?

```
LC -> x3005                .ORIG    x3005
      x3006                LEA      R2, DATA
      x3007 LOOP           LDR      R4, R2, #0
      x3008                ADD      R4, R4, #-3
      x3009                BRzp     LOOP
      x300A DATA           TRAP     x25
                           .FILL    x000B
                           .END
```

Generate Machine Code: Write each line of instruction replacing labels into corresponding addresses.

Preprocess

Build Symbol Table

Generate Machine Code

Example

How does the assembly process work on this code?

```
                    .ORIG     x3005
      x3005          LEA      R2, DATA
      x3006          LDR      R4, R2, #0
      x3007 LOOP     ADD      R4, R4, #-3
      x3008          BRzp     LOOP
      x3009          TRAP     x25
LC -> x300A DATA     .FILL     x000B
                    .END
```

x3005	LEA R2, DATA	E404
x3006	LDR R4, R2, #0	6880
x3007	ADD R4, R4, #-3	193D
x3008	BRzp LOOP	07FE
x3009	TRAP x25	F025
x300A	DATA .FILL x000B	000B

Preprocess Build Symbol Table Generate Machine Code

Example

How does the assembly process work on this code?

x3005	LEA R2, DATA	E404	1110 010 000000100
x3006	LDR R4, R2, #0	6880	0110 100 010 000000
x3007	ADD R4, R4, #-3	193D	0001 100 100 1 11101
x3008	BRzp LOOP	07FE	0000 011 111111110
x3009	TRAP x25	F025	1111 0000 0010 0101
x300A	DATA .FILL x000B	000B	0000 0000 0000 1011

Preprocess Build Symbol Table Generate Machine Code

The importance of two-pass assembly process

Introduction to Computer Architecture and Systems

From Advanced Control Flows to Abstract Data Structures

Fengkai Wang(Iori Oikawa), Fall 2025 @ Drew University

Previously of CSCI 260…

- Assembly Process
- Preprocess + 2-Pass Assembly Process
- The importance of Assembly Process: Another Intermediate Layer for
- Separating Machine Code and Human Readable Code
- Room for Debugging and Optimization
- Opens up Workflow for Interpreters and Compilers

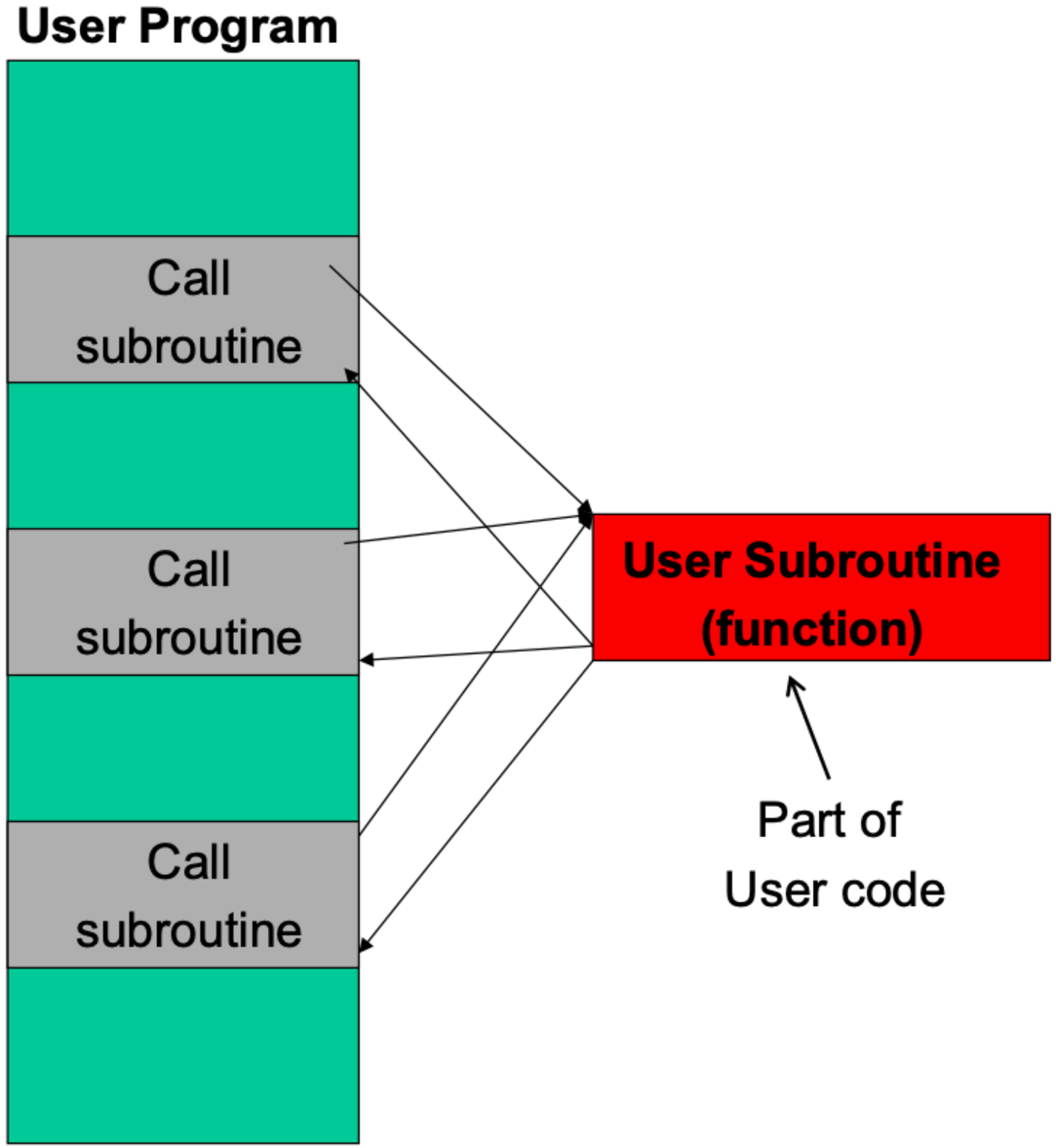
Trap Service Routine and Interrupt Service Routine

TRAP Routine

- System calls to process I/O (or other system tasks)
- Written by system, called by user
- Resides as part of system code
- Steps: Call, Process, Return

TRAP Routine

- System calls to process I/O (or other system tasks)
- Written by system, called by user
- Resides as part of system code
- Steps: Call, Process, Return



Interrupt Routine

- An Interrupt Service Routine is just code to handle an interrupt.
- The LC-3 has only two things that can cause interrupts, the console and an exception.
- The interrupt vector table is located from x100 - x1FF and like the TVT contains the addresses of the code to handle them.
- For the console that would be x180, and for the two types of exceptions x100 and x101 (privilege mode exception, and illegal opcode respectively).

Subtask and Subroutine

Subtasks are…

- A smaller logical unit of work that is decomposed from the main task.
- It may be implemented as a subroutine, but it can also be a directly executed segment of logic within the main program.
- NOT correspond to specific instructions or a formal calling mechanism.

When writing more complex programs (such as performing summation, string processing, or I/O control), programmers often divide the main task into several subtasks, each of which can then be implemented by one or more subroutines.

Subroutines are…

- Blocks of code can be encoded as subroutines
  - Contain one entry point – one exit point each
- A subroutine is a program fragment that…
  - Lives in user space
  - Performs a well-defined task
  - Is invoked (called) by a user program
  - Returns control to the calling program when finished

Reasons for subroutines

- Reuse useful (and debugged!) code without having to keep typing it in
- Divide task among multiple programmers
- Use vendor-supplied library of useful routines

Elements for Subroutine R7, JSR, JSRR, RET

Register R7

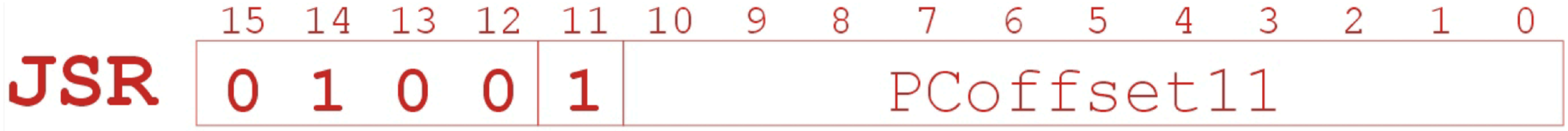
R7 is nothing special, but sometimes stores address from JSR(R)

R7

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

JSR

Like Jump, but saves PC in R7.



- Jumps to a location (like an unconditional branch), and saves current PC (address of next instruction) in R7
- Saving the return address is called “linking”
- Target address is PC-relative (PC + Sext(IR[10:0]))
- Bit 11 specifies addressing mode
  - If =1, PC-relative: target address = PC + Sext(IR[10:0])
  - If =0, register: target address = contents of Register[IR[8:6]] (JSRR)

JSRR

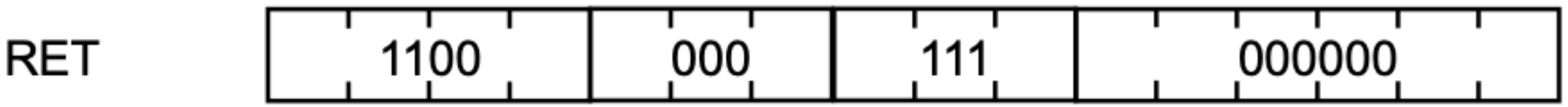
Like Jump, but saves PC in R7.



- Just like JSR, except Register addressing mode.
- Target address is Base Register
- Bit 11 specifies addressing mode, now is 0.

What important feature does JSRR provide that JSR does not?

RET = JMP R7



Example: JSR

```
        .ORIG x3000
        JSR PRINT_HELLO      ; Call Subroutine
        HALT                 ; End of Main Program

PRINT_HELLO
        LEA R0, HELLO_STR    ; R0 ← Begin Address
        (TRAP) PUTS          ; Output String
        RET                  ; Return to Main

HELLO_STR .STRINGZ "Hello, LC-3!"
        .END
```

Example: JSRR

```
        .ORIG x3000
        LD R3, SUB_ADDR      ; R3 ← Address to Subroutine
        JSRR R3              ; Referring Subroutine
        HALT

SUB_ADDR .FILL PRINT_HELLO ; Place to store address

; Definition of Subroutine
PRINT_HELLO
        LEA R0, MSG
        PUTS
        RET

MSG .STRINGZ "Hello via JSRR!"
        .END
```

Using Subroutines: Insert your snippets

Function Call with 2's Complement

```
2sComp:
    NOT R0, R0      ; flip bits
    ADD R0, R0, #1  ; add 1
    RET             ; return to caller
```

- To call from a program (within 1024 instructions):

```
JSR 2sComp
```

```
; need to compute
; R4 = R1 - R3
ADD R0, R3, #0      ; R0 ← R3
JSR 2sComp          ; negate
ADD R4, R1, R0      ; add to R1
```

Note: Caller should save R0 if it will need it later!

Saving and Restoring Registers

Must save the value of a register if

- Its value will be destroyed by service routine, AND
- We will need to use the value after that action.

But wait… WHO SAVES THE REGISTERS?

WHO SAVES THE REGISTERS

Is that Caller of service routine?

- It knows what it needs later, but may not know what gets altered by called routine

Is Called service routine?

- It knows what it alters, but does not know what will be needed later by calling routine

Answer: Caller or Callee

The Calling routine -- “caller-save”

- Save all registers that will be needed later
- In special cases, avoid using those registers altogether

The Called routine -- “callee-save”

- Before start, save any registers that will be altered (unless altered value is desired by calling program!)
- Before return, restore those same registers

Takeaway: Caller / Callee Saves

- Values are saved by storing them in memory.
- Generally use “callee-save” strategy, except for return values.
- Remember to save R7before any other call (incl. TRAPs) or you won’t be able to return…

Argument and Return Values

Arguments

- A value passed in to a subroutine is called an argument.
- This is a value needed by the subroutine to do its job.

For Example:

- In 2sComp routine, R0 is the number to be negated
- In PUTS routine counting occurrences of characters, R0 is address of string to be printed.

Return Values

- A value passed out of a subroutine is called a return value.
- This is the value that you called the subroutine to compute.

For Example:

- In 2sComp routine, negated value is returned in R0
- In PUTS routine counting occurrences of characters, character read from the keyboard is returned in R0.

Using Subroutines

In order to use a subroutine, a programmer must know…

- Its address (or the label that will be bound to its address)
- Its function (what does it do?)
- Its arguments (where to pass data in, if any)
- Its return values (where to get computed data, if any)

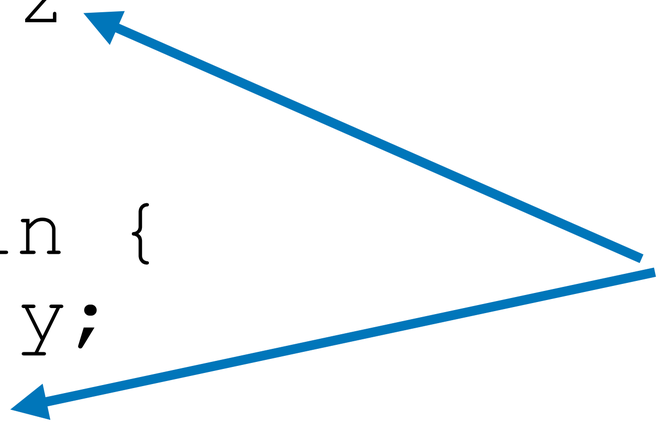
NOTE: The programmer does not need to know how the subroutine works, but what changes are visible in the machine's state after the routine has run.

Application in High Level Language: Scope

Example in C

```
int sub(x, y){
    int z
….}

int main {
int x, y;
int z;
…
z = sub(x, y);
…}
```



These two are Different variables in C

BUT Same (registers) in assembly!

Exercise

Submit your code in .asm file before Sunday (23:50) as the Quiz

- Write a program that checks the initial value in memory location x4000 to see if it is a valid ASCII code.
- If it is a valid ASCII code, prints the character.
- If the value in x4000 is not a valid ASCII code, the program prints nothing.

Quiz:

Back Side: Submit your card to me Next Monday

- Look up ‘Pushdown Automata(PDA)’ on the internet, write a short paragraph to describe the relationship between PDA and stack.

Next Up:
Abstract Data Structures:
Stack and Queue

Introduction to Computer Architecture and Systems

Abstract Data Structures

Fengkai Wang(Iori Oikawa), Fall 2025 @ Drew University

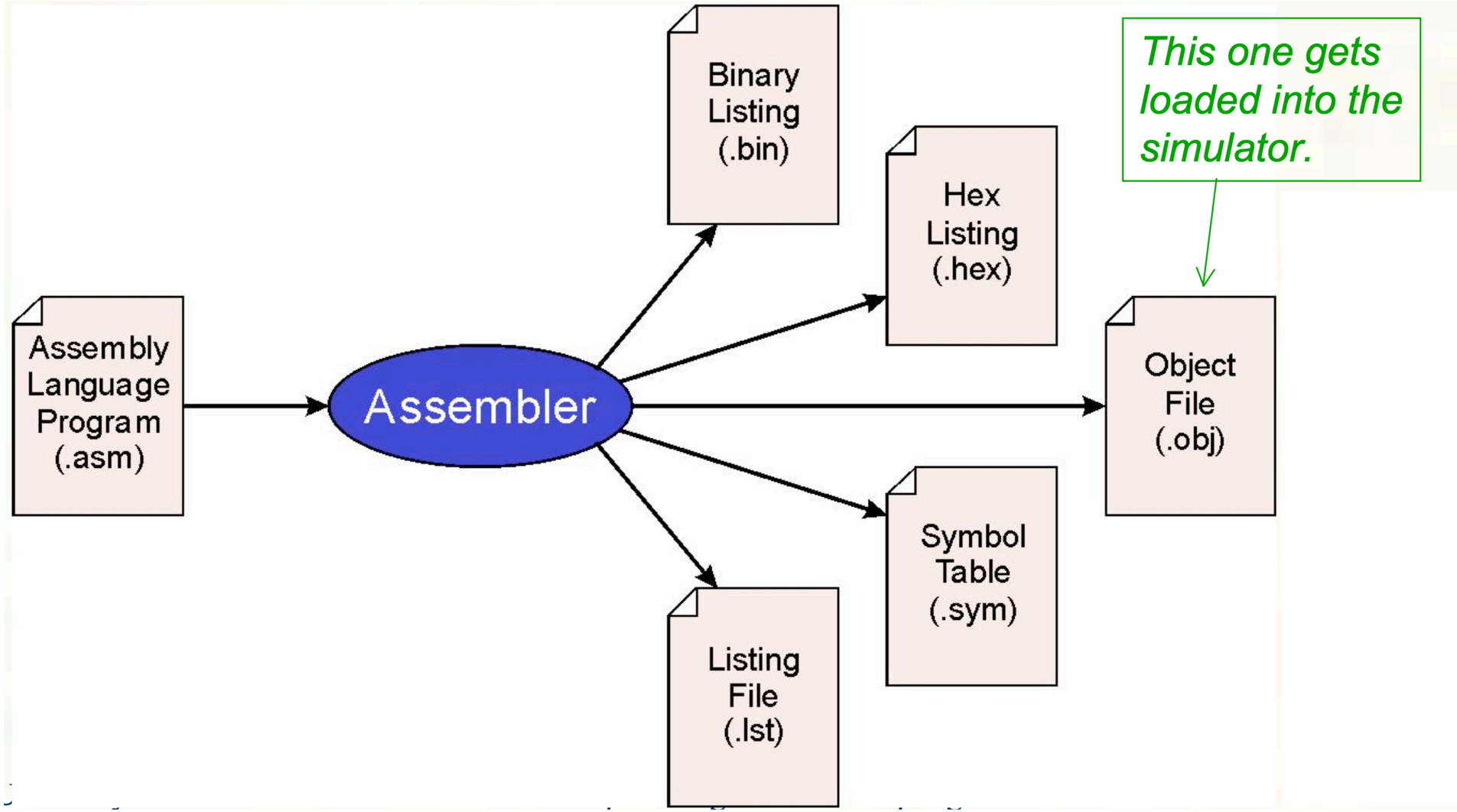
Previously of CSCI 260…

- Subroutine and Subtask
- Service Routine: Trap and Interrupt
- Elements of Subroutine: JSR(R), R7, RET
- Using Subroutine
- Caller and Callee Saved Subroutine

LC-3 Assembler

LC-3 Assembler

Turning Assembly Language Program to Executable Image



Object File

Contains starting address (location where program must be loaded), followed by machine instructions.

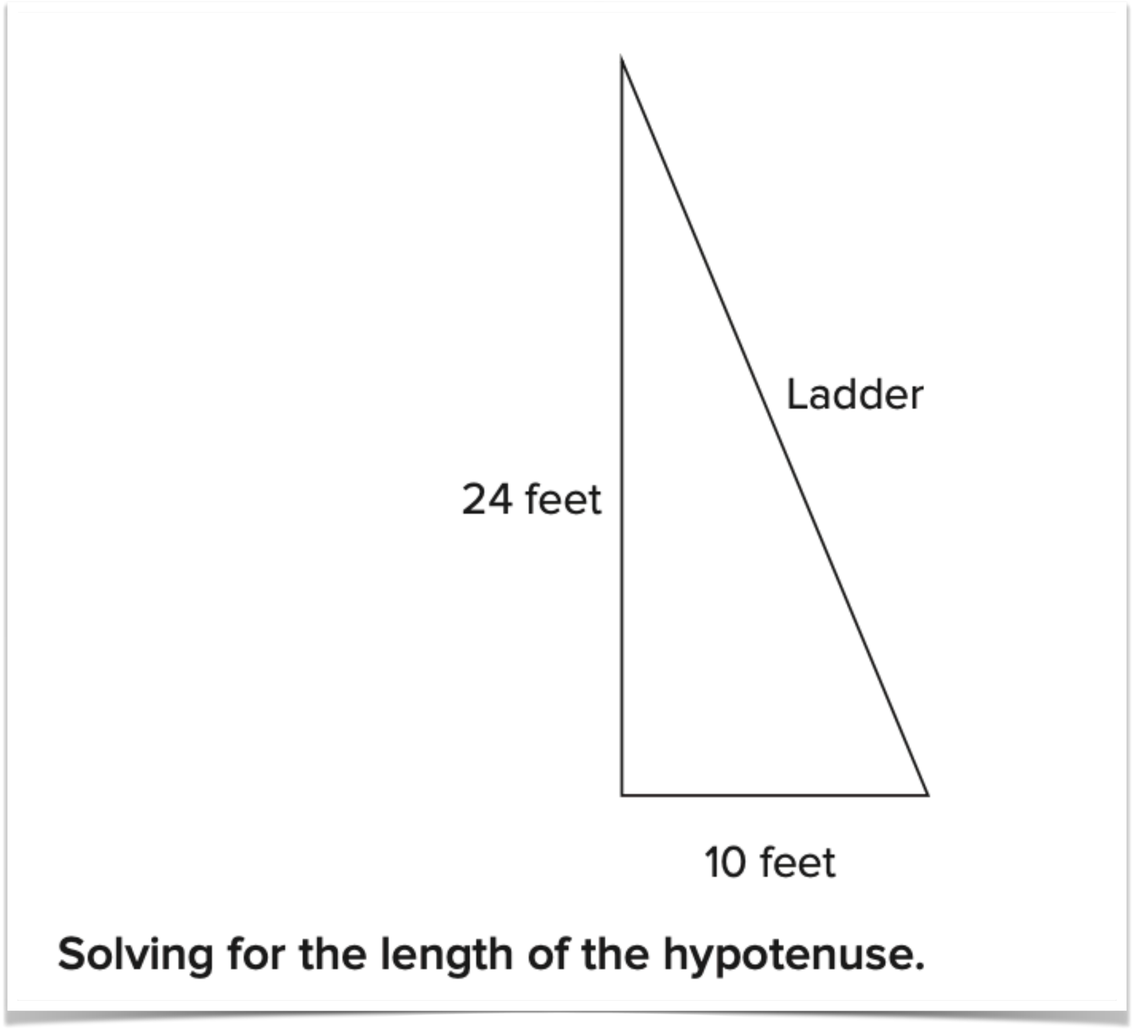
- Example: The Beginning of Count Character's Object File

```
0011000000000000        .ORIG x3000
0101010010100000        AND R2, R2, #0
0010011000010001        LD R3, PTR
1111000000100011        TRAP x23
```

Add Multiple Object File

- An object file is not necessarily a complete program.
  - system-provided library routines
  - code blocks written by multiple developers
- For LC-3 simulator, can load multiple object files into memory, then start executing at a desired address.
  - system routines, such as keyboard input, are loaded automatically
    - loaded into “system memory,” below x3000
    - user code should be loaded between x3000 and xFDFF
  - each object file includes a starting address
  - be careful not to load overlapping object files

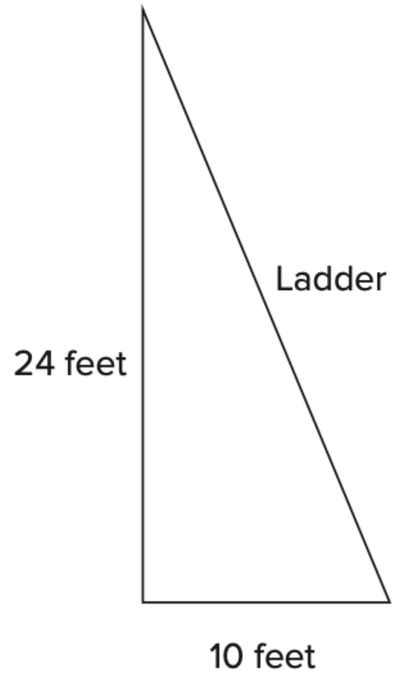
New Pseudo-Op: .EXTERNAL



Solving for the length of the hypotenuse.

Example: Solving the length of the hypotenuse, calculate square root.

```
01              ...
02              ...
03              LD      R0,SIDE1
04              BRz     S1
05              JSR     SQUARE
06      S1      ADD     R1,R0,#0
07              LD      R0,SIDE2
08              BRz     S2
09              JSR     SQUARE
0A      S2      ADD     R0,R0,R1
0B              JSR     SQRT
0C              ST      R0,HYPOT
0D              BRnzp   NEXT_TASK
0E      SQUARE  ADD     R2,R0,#0
0F              ADD     R3,R0,#0
10      AGAIN   ADD     R2,R2,#-1
11              BRz     DONE
12              ADD     R0,R0,R3
13              BRnzp   AGAIN
14      DONE    RET
15      SQRT    ...             ; R0 <-- SQRT(R0)
16              ...             ;
17              ...             ; How do we write this subroutine?
18              ...             ;
19              ...             ;
1A              RET
1B      SIDE1   .BLKW   1
1C      SIDE2   .BLKW   1
1D      HYPOT   .BLKW   1
1E              ...
1F              ...
```



Solving for the length of the hypotenuse.

```
01                ...
02                ...
03                LD      R0,SIDE1
04                BRz     S1
05                JSR     SQUARE
06      S1        ADD     R1,R0,#0
07                LD      R0,SIDE2
08                BRz     S2
09                JSR     SQUARE
0A      S2        ADD     R0,R0,R1
0B                JSR     SQRT
0C                ST      R0,HYPOT
0D                BRnzp   NEXT_TASK
0E      SQUARE    ADD     R2,R0,#0
0F                ADD     R3,R0,#0
10      AGAIN     ADD     R2,R2,#-1
11                BRz     DONE
12                ADD     R0,R0,R3
13                BRnzp   AGAIN
14      DONE      RET
15      SQRT      ...
16                ...
17                ...
18                ...
19                ...
1A                RET
1B      SIDE1     .BLKW   1
1C      SIDE2     .BLKW   1
1D      HYPOT     .BLKW   1
1E                ...
1F                ...
```

```
01                ...
02                ...
03                .EXTERNAL SQRT
04                ...
05                ...
06                LD      R0,SIDE1
07                BRz     S1
08                JSR     SQUARE
09      S1        ADD     R1,R0,#0
0A                LD      R0,SIDE2
0B                BRz     S2
0C                JSR     SQUARE
0D      S2        ADD     R0,R0,R1 ; R0 contains argument x
0E                LD      R4,BASE
0F                JSRR    R4
10                ST      R0,HYPOT
11                BRnzp   NEXT_TASK
12      SQUARE    ADD     R2,R0,#0
13                ADD     R3,R0,#0
14      AGAIN     ADD     R2,R2,#-1
15                BRz     DONE
16                ADD     R0,R0,R3
17                BRnzp   AGAIN
18      DONE      RET
19      BASE      .FILL   SQRT
1A      SIDE1     .BLKW   1
1B      SIDE2     .BLKW   1
1C      HYPOT     .BLKW   1
1D                ...
1E                ...
```

BASE contains the starting address of SQRT subroutine

SQRT is available in Math Library

Linking and Loading

Loading

…the process of copying an executable image into memory

More sophisticated loaders are able to relocate images to fit into available memory.

Must readjust branch targets, load/store addresses during Loading.

Linking

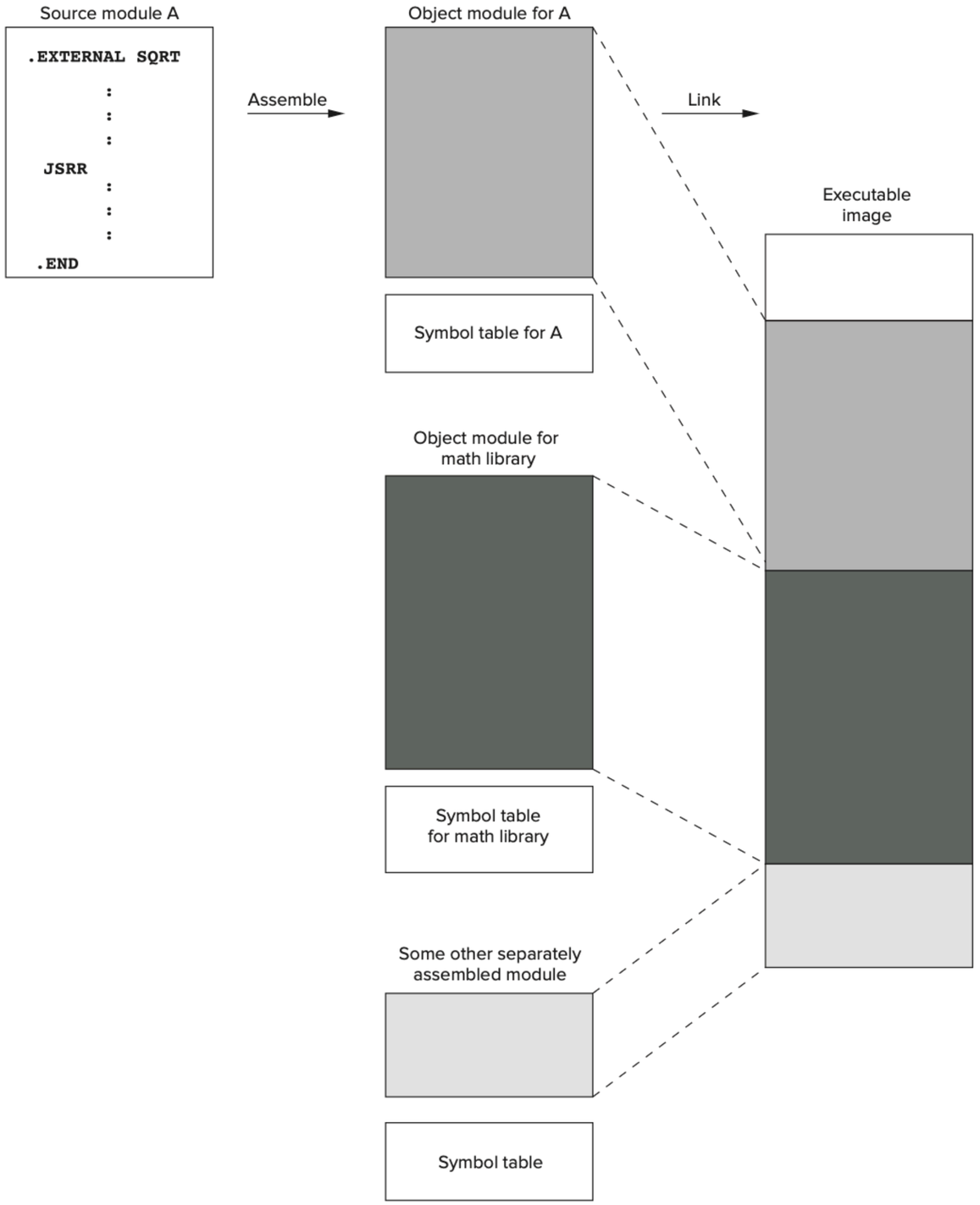
…the process of resolving symbols between independent object files

Suppose we define a symbol in one module, and want to use it in another…

Some notation, such as .EXTERNAL, is used to tell assembler that a symbol is defined in another module…

The linker will search symbol tables of other modules to resolve symbols and complete code generation before loading.

Assembly Across Files



Argument and Return Values

Arguments

- A value passed in to a subroutine is called an argument.
- This is a value needed by the subroutine to do its job.

For Example:

- In 2sComp routine, R0 is the number to be negated
- In PUTS routine counting occurrences of characters, R0 is address of string to be printed.

Return Values

- A value passed out of a subroutine is called a return value.
- This is the value that you called the subroutine to compute.

For Example:

- In 2sComp routine, negated value is returned in R0
- In PUTS routine counting occurrences of characters, character read from the keyboard is returned in R0.

Using Subroutines

In order to use a subroutine, a programmer must know…

- Its address (or the label that will be bound to its address)
- Its function (what does it do?)
- Its arguments (where to pass data in, if any)
- Its return values (where to get computed data, if any)

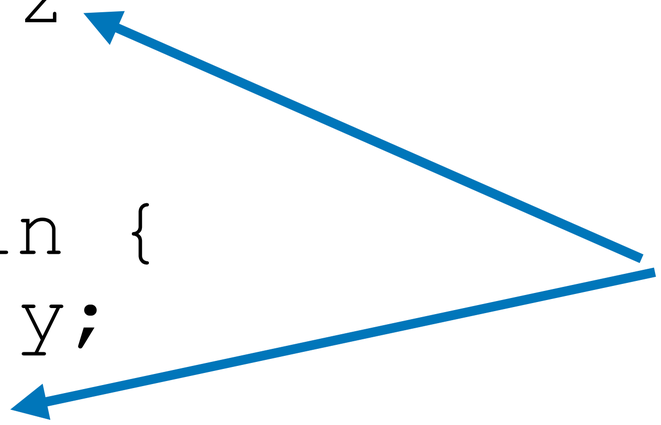
NOTE: The programmer does not need to know how the subroutine works, but what changes are visible in the machine's state after the routine has run.

Application in High Level Language: Scope

Example in C

```
int sub(x, y){
    int z
….}

int main {
int x, y;
int z;
…
z = sub(x, y);
…}
```



These two are Different variables in C

BUT Same (registers) in assembly!

Exercise

Submit your code in .asm file before Wednesday (23:50) as the Quiz

- Write a program that checks the initial value in memory location x4000 to see if it is a valid ASCII code.
- If it is a valid ASCII code, prints the character.
- If the value in x4000 is not a valid ASCII code, the program prints nothing.

Quiz:

Back Side

- Look up ‘Pushdown Automata(PDA)’ on the internet, write a short paragraph to describe the relationship between PDA and stack.

Abstract Data Structure

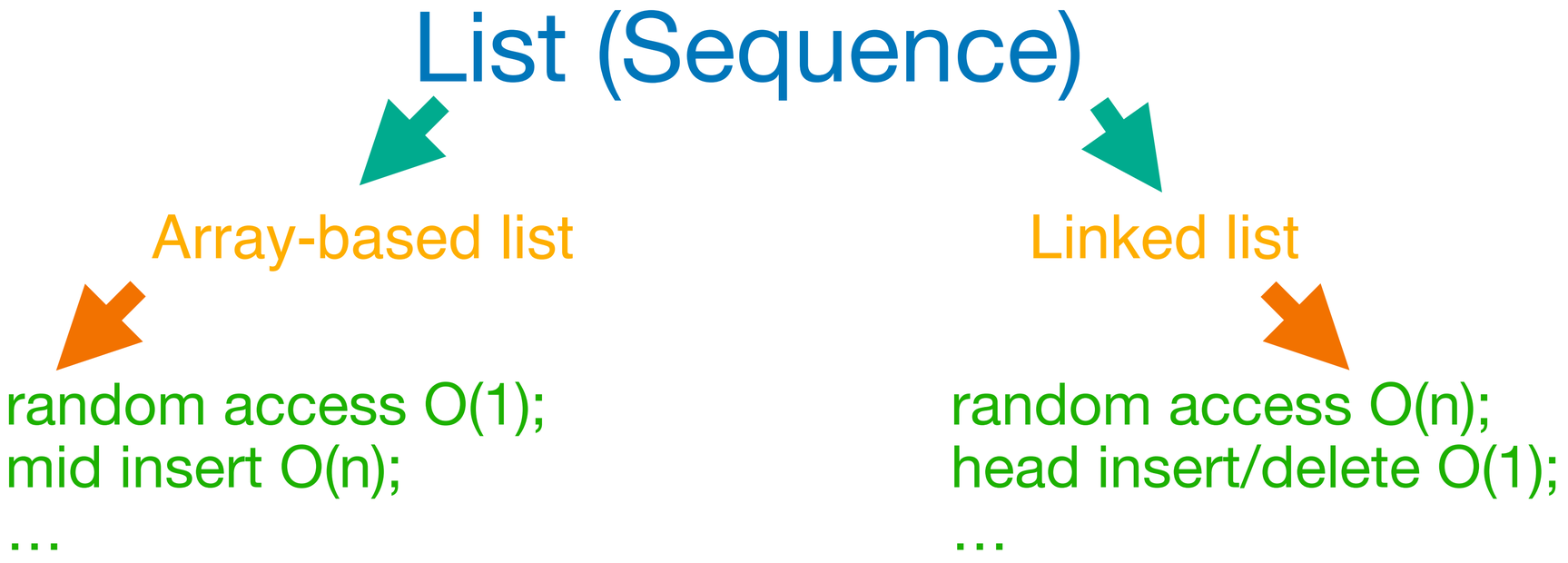
A behavior contract: operations + semantics

Abstract Data Structure: WHY?

Example…

List (Sequence)

Example…



“Stack / Queue / Set / Graph” are universal concepts
We can switch languages or libraries with the same mental model
Which lowers onboarding & migration costs

What it does ≠ How it is done

Stack and Queue

Stack

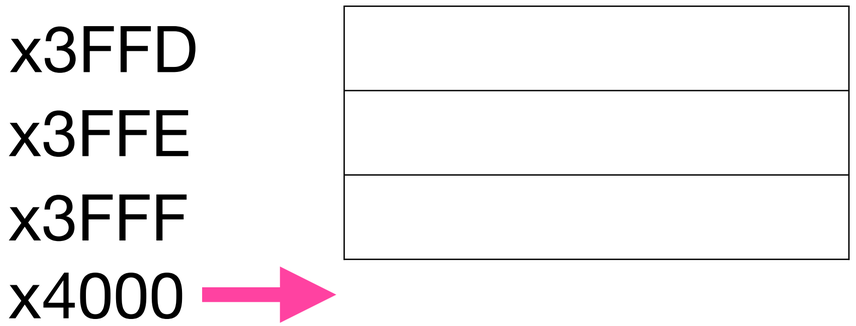
Why Stacks?

First In, First Out

- Used in ASCII to Binary etc.
- Interrupt processing, system/function calls (System Call Stack)
- Need it to support high level languages

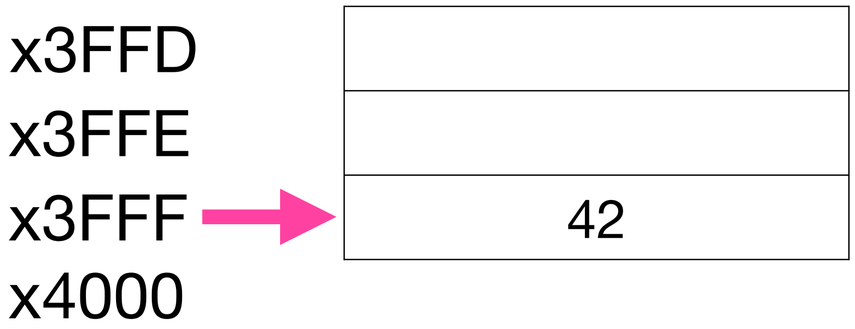
What does a stack do?

→ : points the top of stack



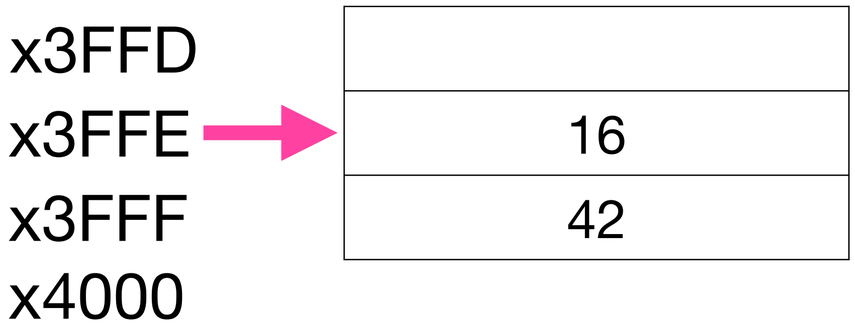
What does a stack do?

→ : points the top of stack



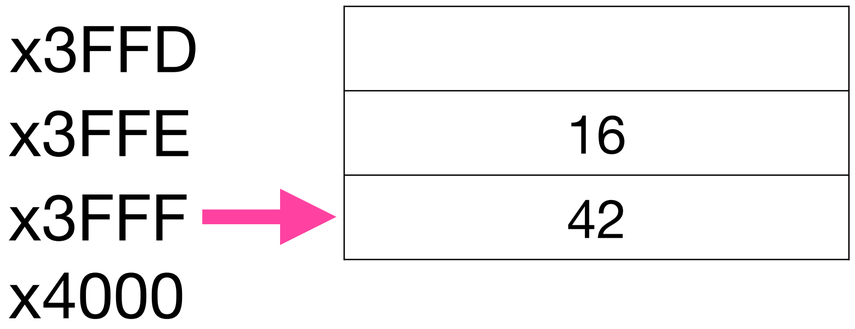
What does a stack do?

→ : points the top of stack



What does a stack do?

→ : points the top of stack



Implement a Stack

You need a Pointer and PUSH/POP operators.

Register R6

R7 is nothing special, but stores the top of stack.

R6

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Implement a Stack

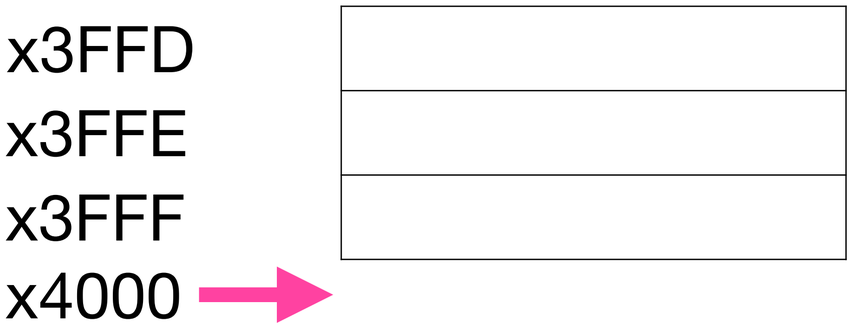
You need a Pointer, and PUSH/POP operators.

Stack Pointer: Register R6

R6 is assigned to store the top of stack.

R6

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0



At initial state, R6 is pointing x4000.

Implement a Stack

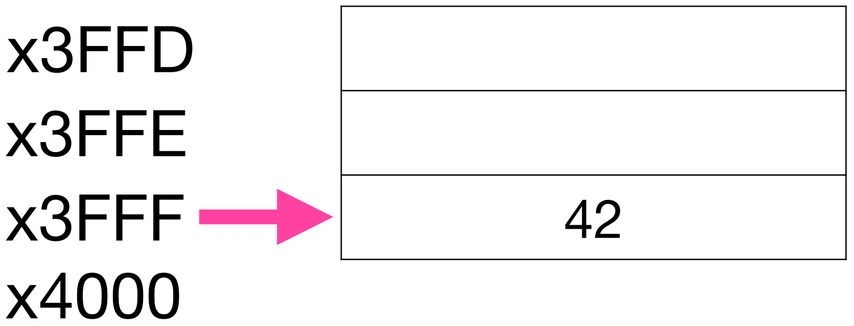
You need a Pointer, and PUSH/POP operators.

Stack Pointer: Register R6

R6 is assigned to store the top of stack.

R6

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	1	1	1	1	1	1	1	1	1	1	1	1	1	1



R6 is grows when stack grows by PUSH.

Implement a Stack

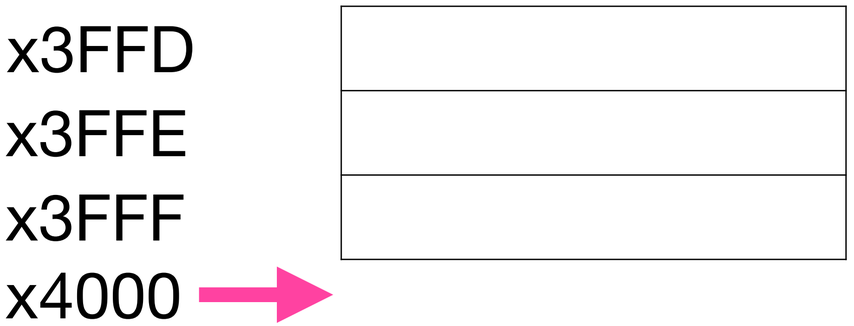
You need a Pointer, and PUSH/POP operators.

Stack Pointer: Register R6

R6 is assigned to store the top of stack.

R6

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0



And moves back when we POPing the stack.

Implement a Stack

You need a Pointer, and PUSH/POP operators.

Stack Operations: PUSH

PUSH is defined by us. Treat it like a label.

```
PUSH    ADD  R6, R6, #-1
        STR  R0, R6, #0
```

If you pre load number 42 in R0, x3FFF will contain #42 after this push.

And R6 will contain x3FFF, as it is the new top of stack.

Implement a Stack

You need a Pointer, and PUSH/POP operators.

Stack Operations: POP

POP is defined by us. Treat it like a label.

```
POP         LDR   R0, R6, #0
            ADD   R6, R6, #1
```

After the POP, the value of the top stack will be in R0.

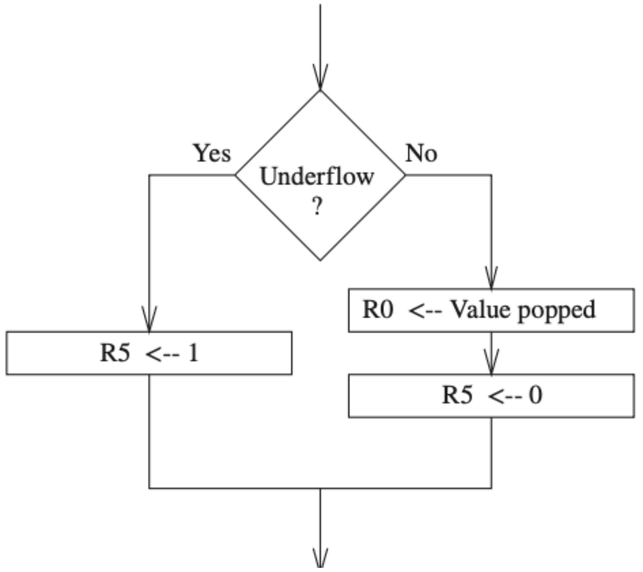
And R6 will contain x4000, as it is the new top of stack.

Underflow and Overflow

Underflow: Over-drafting Value from a Stack

We could do some error handling for underflow like so…

```
POP     LD      R1,EMPTY
        ADD     R2,R6,R1        ; Compare stack
        BRz     UNDERFLOW       ; pointer with x4000.
;
        LDR     R0,R6,#0
        ADD     R6,R6,#1
;
        RET
EMPTY   .FILL   xC000           ; EMPTY <-- negative of x4000
```

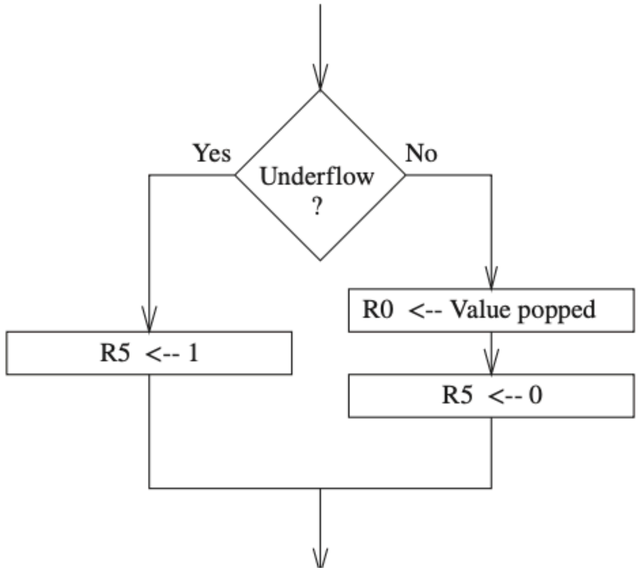


Underflow and Overflow

Underflow: Over-drafting Value from a Stack

We could do some error handling for underflow like so…

```
POP     LD      R1,EMPTY
        ADD     R2,R6,R1        ; Compare stack
        BRz     UNDERFLOW       ; pointer with x4000.
;
        LDR     R0,R6,#0
        ADD     R6,R6,#1
;
        RET
EMPTY   .FILL   xC000           ; EMPTY <-- negative of x4000
```



Underflow and Overflow

Underflow: Over-drafting Value from a Stack

```
POP       AND     R5,R5,#0
          LD      R1,EMPTY
          ADD     R2,R6,R1
          BRz     Failure
          LDR     R0,R6,#0
          ADD     R6,R6,#1
          RET
Failure   ADD     R5,R5,#1
          RET
EMPTY     .FILL   xC000          ; EMPTY <-- -x4000
```

- Upon return from the POP routine, the calling program would examine R5 to determine whether the POP completed successfully (R5=0), or not (R5=1).
- Note that since the POP routine reports success or failure in R5, whatever was stored in R5 before the POP routine was called is lost. Thus, it is the job of the calling program to save the contents of R5 before the JSR instruction is executed if the value stored there will be needed later. This this is an example of a caller-save situation.

Underflow and Overflow

Overflow: Pushing too many values when there's no space.

We could do some error handling for overflow like so…

```
PUSH        LD      R1,MAX
            ADD     R2,R6,R1
            BRz     OVERFLOW
;
            ADD     R6,R6,#-1
            STR     R0,R6,#0
;
            RET
MAX         .FILL   xC005           ; MAX <-- negative of x3FFB
```

Underflow and Overflow

Overflow: Pushing too many values when there's no space.

```
PUSH        AND     R5,R5,#0
            LD      R1,MAX
            ADD     R2,R6,R1
            BRz     Failure
            ADD     R6,R6,#-1
            STR     R0,R6,#0
            RET
Failure     ADD     R5,R5,#1
            RET
MAX         .FILL   xC005           ; MAX <-- -x3FFB
```

- We augment the PUSH routine with instructions to store 0 (success) or 1 (failure) in R5, depending on whether or not the push completed success- fully. Upon return from the PUSH routine, the calling program would examine R5 to determine whether the PUSH completed successfully (R5= 0) or not (R5= 1).
- Note again that since the PUSH routine reports success or failure in R5, we have another example of a caller-save situation. That is, since whatever was stored in R5 before the PUSH routine was called is lost, it is the job of the calling pro- gram to save the contents of R5 before the JSR instruction is executed if the value stored in R5 will be needed later.

Conclusion

Stack Protocol

- The POP and PUSH routines allow us to use memory locations x3FFF through x3FFB as a five-entry stack.
- If we wish to push a value onto the stack, we simply load that value into R0 and execute JSR PUSH.
- To pop a value from the stack into R0, we simply execute JSR POP.
- If we wish to change the location or the size of the stack, we adjust BASE and MAX accordingly.

```
01   ;
02   ; Subroutines for carrying out the PUSH and POP functions.  This
03   ; program works with a stack consisting of memory locations x3FFF
04   ; through x3FFB.  R6 is the stack pointer.
05   ;
06   POP             AND     R5,R5,#0        ; R5 <-- success
07                   ST      R1,Save1        ; Save registers that
08                   ST      R2,Save2        ; are needed by POP
09                   LD      R1,EMPTY        ; EMPTY contains -x4000
0B                   ADD     R2,R6,R1        ; Compare stack pointer to x4000
0C                   BRz     fail_exit       ; Branch if stack is empty
0D   ;
0E                   LDR     R0,R6,#0        ; The actual "pop"
0F                   ADD     R6,R6,#1        ; Adjust stack pointer
10                   BRnzp   success_exit
11   ;
12   PUSH            AND     R5,R5,#0
13                   ST      R1,Save1        ; Save registers that
14                   ST      R2,Save2        ; are needed by PUSH
15                   LD      R1,FULL         ; FULL contains -x3FFB
16                   ADD     R2,R6,R1        ; Compare stack pointer to x3FFB
17                   BRz     fail_exit       ; Branch if stack is full
18   ;
19                   ADD     R6,R6,#-1       ; Adjust stack pointer
1A                   STR     R0,R6,#0        ; The actual "push"
1B   success_exit    LD      R2,Save2        ; Restore original
1C                   LD      R1,Save1        ; register values
1D                   RET
1E   ;
1F   fail_exit       LD      R2,Save2        ; Restore original
20                   LD      R1,Save1        ; register values
21                   ADD     R5,R5,#1        ; R5 <-- failure
22                   RET
23   ;
24   EMPTY           .FILL   xC000           ; EMPTY contains -x4000
25   FULL            .FILL   xC005           ; FULL contains  -x3FFB
26   Save1           .FILL   x0000
27   Save2           .FILL   x0000
```

Queue

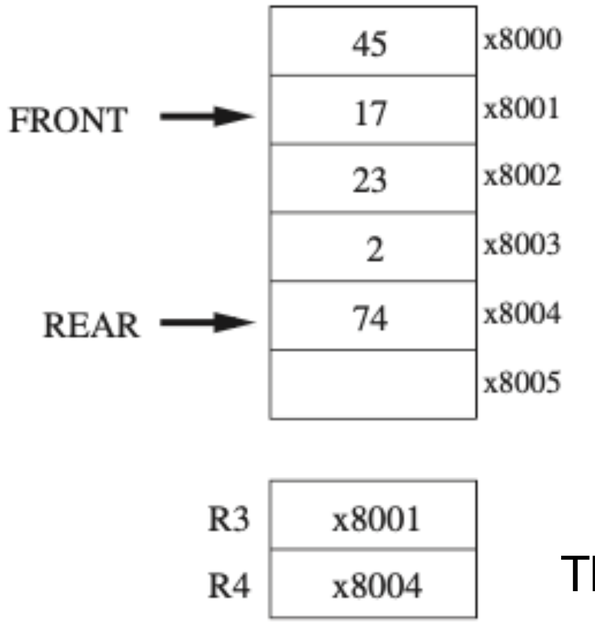
Why Queues?

“First In, Last Out”

- Used for Data Streams
- …and corresponding components like Buffers

What does Queues do?

We arbitrarily assign the FRONT pointer to the location just before the first element of the queue. We assign the REAR pointer to the location containing the most recent element that was added to the queue. Let's use R3 as our FRONT pointer and R4 as our REAR pointer.



We need…

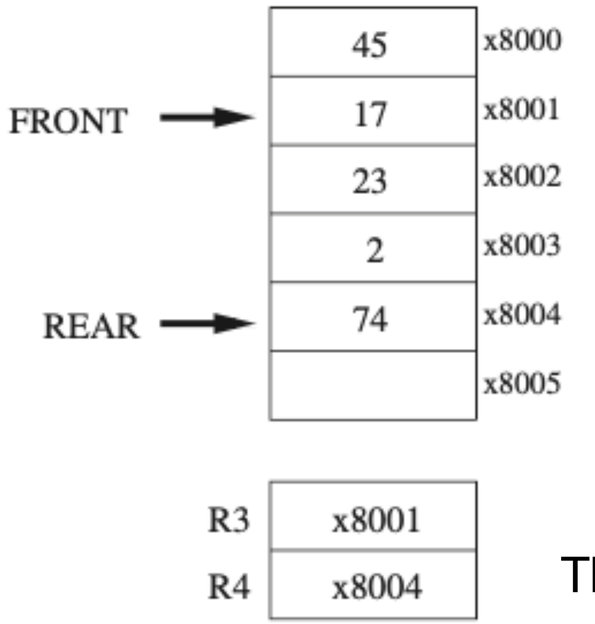
A FRONT pointer for servicing (i.e., removing elements from the front of the queue)

and a REAR pointer for entering (i.e., inserting into the rear of the queue)

This shows a queue in which five values were entered into the queue.

Since FRONT = x8001, the values 45 in memory location x8000 and 17 in x8001 must have been removed, and the front element of the queue is 23, the value contained in x8002.

What does Queues do?



We arbitrarily assign the FRONT pointer to the location just before the first element of the queue.

We assign the REAR pointer to the location containing the most recent element that was added to the queue.

Let's use R3 as our FRONT pointer and R4 as our REAR pointer.

This shows a queue in which five values were entered into the queue.

Since FRONT = x8001, the values 45 in memory location x8000 and 17 in x8001 must have been removed, and the front element of the queue is 23, the value contained in x8002.

Implement a Queue

You need two Pointers, and Remove / Insert operations.

Queue Operations: REMOVE(from FRONT)

REMOVE is defined by us. Treat it like a label.

```
REMOVE      ADD   R3, R3, #1
            LDR   R0, R3, #0
```

After the REMOVE, the value 23 at x8002 stack will be in R0.

And R3 will contain x8002, as it is address of the new FRONT.

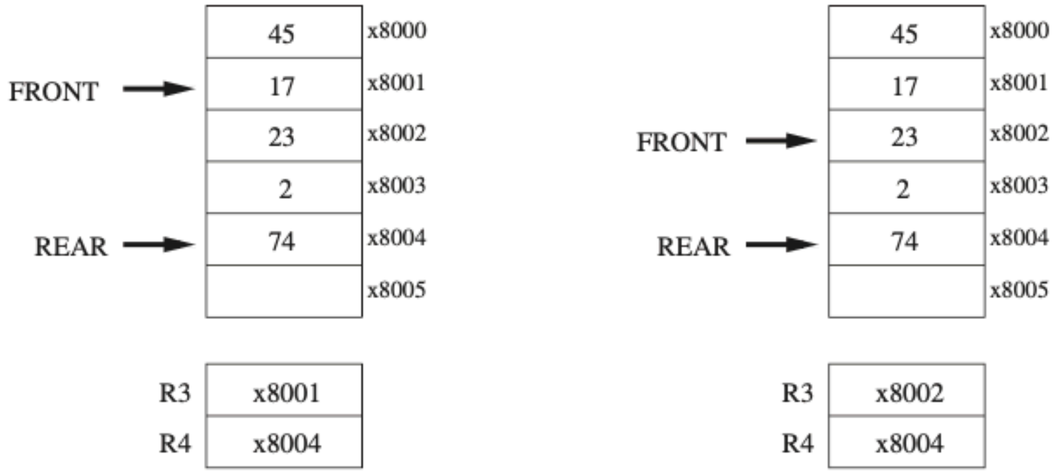
Implement a Queue

You need two Pointers, and Remove / Insert operations.

Queue Operations: REMOVE(from FRONT)

REMOVE is defined by us. Treat it like a label.

```
REMOVE      ADD   R3, R3, #1
            LDR   R0, R3, #0
```



After the REMOVE, the value 23 at x8002 stack will be in R0.

And R3 will contain x8002, as it is address of the new FRONT.

Implement a Queue

You need two Pointers, and Remove / Insert operations.

Queue Operations: INSERT(from BACK)

INSERT is defined by us. Treat it like a label.

```
INSERT    ADD  R4, R4, #1
          STR  R0, R4, #0
```

If you pre load number 10 in x8005, it will be added to the queue.

And R4 will contain x8005, as it is the new tail of the queue.

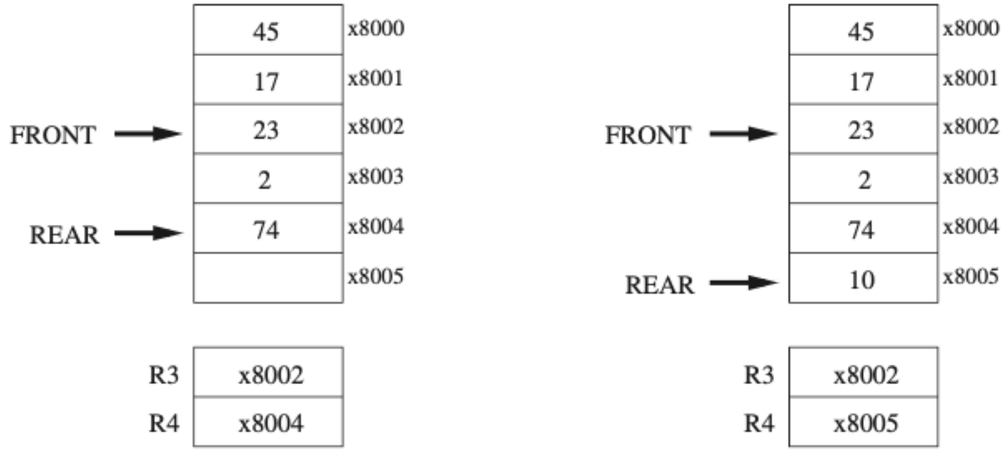
Implement a Queue

You need two Pointers, and Remove / Insert operations.

Queue Operations: INSERT(from BACK)

INSERT is defined by us. Treat it like a label.

```
INSERT    ADD   R4, R4, #1
          STR   R0, R4, #0
```



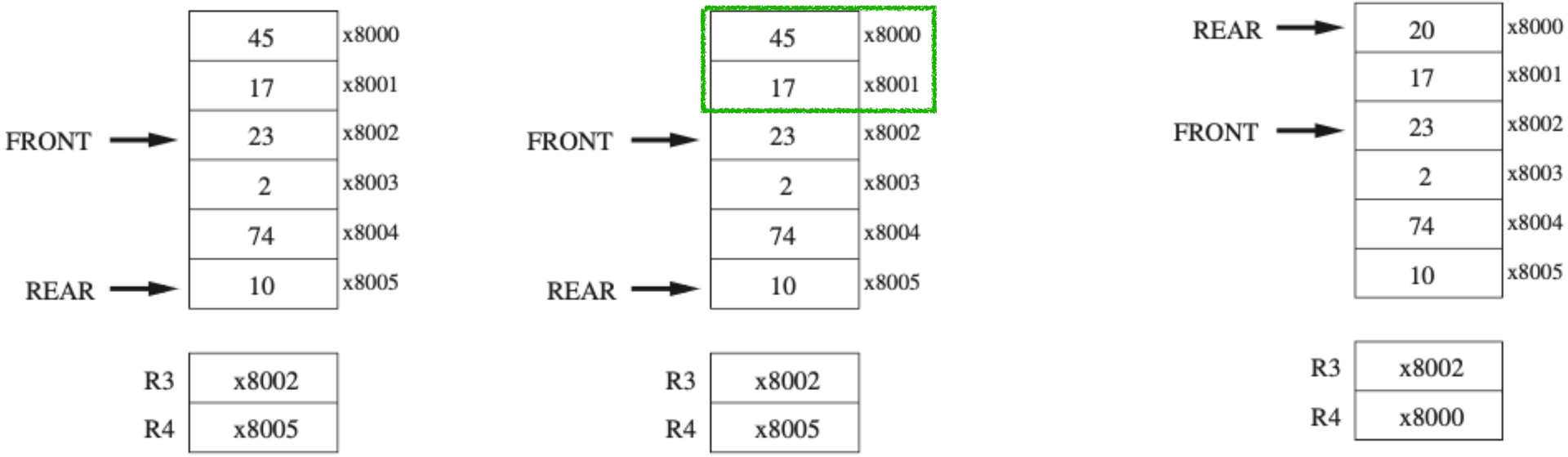
If you pre load number 10 in x8005, it will be added to the queue.

And R4 will contain x8005, as it is the new tail of the queue.

Wait… Can we add more elements to the queue?

Wrap-Around

Allowing the available storage location to store new data.



- We could insert 20 safely to x8000, since it is safe to insert values.
- Notice REAR pointer is wrapping around the memory.

New REMOVE with Wrap-Around

```
            LD   R2, LAST
            ADD  R2,R3,R2
            BRnp SKIP_1
            LD   R3,FIRST
            BR   SKIP_2
SKIP_1      ADD  R3,R3,#1
SKIP_2      LDR  R0,R3,#0 ; R0 gets the front of the queue
            RET
LAST        .FILL x7FFB   ; LAST contains the negative of 8005
FIRST       .FILL x8000
```

- If we wish to remove, we must first see if FRONT contains the address x8005. If it does, the front of the queue is in x8000.

New INSERT with Wrap-Around

```
            LD   R2, LAST
            ADD  R2,R4,R2
            BRnp SKIP_1
            LD   R4,FIRST
            BR   SKIP_2
SKIP_1      ADD  R4,R4,#1
SKIP_2      STR  R0,R4,#0 ; R0 gets the front of the queue
            RET
LAST        .FILL 7FFB    ; LAST contains the negative of 8005
FIRST       .FILL x8000
```

- If REAR contains x8005, we need to set R4 to x8000 before we can insert an element at the rear of the queue.

Underflow and Overflow

Underflow: Removing Value from an empty Queue

```
                    AND   R5,R5,#0  ; Initialize R5 to 0
                    NOT   R2,R3
                    ADD   R2,R2,#1  ; R2 contains negative of R3
                    ADD   R2,R2,R4
                    BRz   UNDERFLOW
                    ; code to remove the front of the queue and return success.
UNDERFLOW           ADD   R5,R5,#1
                    RET
```

- Notice if FRONT = REAR, the queue is empty.
- We first check to see if the queue is empty, that is, if R3 = R4. If so, we branch to UNDERFLOW, where we set R5 to failure, restore R1, and return.
- If not, carry out the code to remove the front of the queue.

Underflow and Overflow

Overflow: Inserting value to a full Queue.

- To insert an element to the back of the queue, we first increment the REAR pointer.
- If that causes FRONT = REAR, then the queue already contains n-1 elements, which means it is full so we cannot insert any more elements. We decrement the REAR pointer, set R5 to 1, and return.

Conclusion

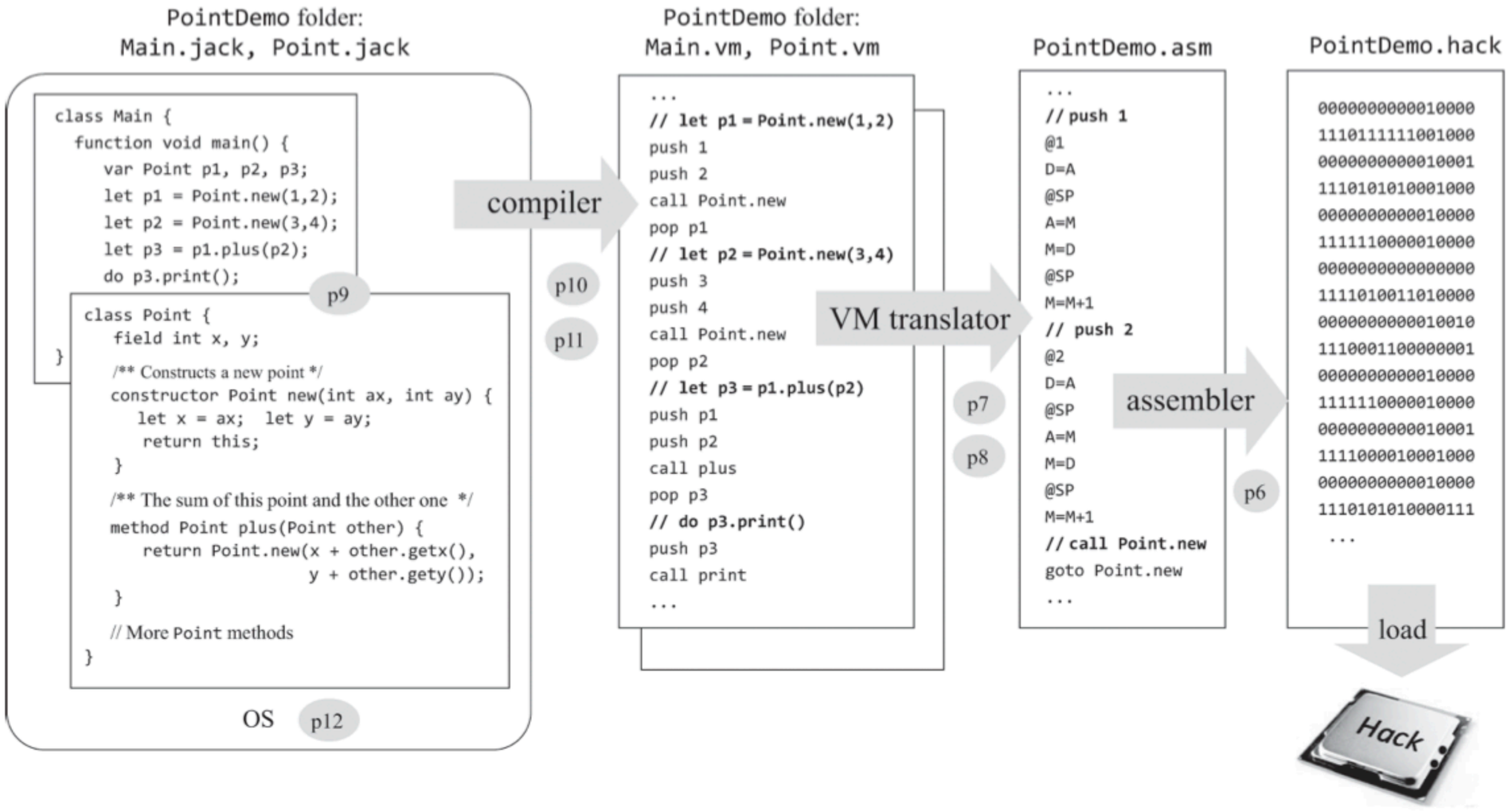
Queue Protocol

- We have allocated locations x8000 to x8005 for our queue, x8000 being the FIRST location and x8005 being the LAST location.
- To insert, we first have to make sure the queue is not full. To do that, we increment the REAR pointer (R4) and then test REAR=FRONT. If the REAR pointer was initially x8005, we increment REAR by setting it to x8000; that is, we need to wrap around. If the queue is full, we need to set REAR back to its original value, and return, reporting failure (R5 = 1). If the queue is not full, we store the item we wish to insert (which is in R0) in REAR, and return, reporting success (R5 = 0).
- To remove, we first make sure the queue is not empty by testing whether REAR=FRONT. If REAR=FRONT, the queue is empty, so we return, reporting failure. If REAR is not the same as FRONT, the queue is not empty, so we can remove the front element. To do this, we first test to see if FRONT=x8005. If it is, we set FRONT=x8000. If it isn't, we increment FRONT. In both cases, we then load the value from that memory location into R0, and return, reporting success.

```
00 ;Input: R0 for item to be inserted, R3 is FRONT, R4 is REAR
01 ;Output: R0 for item to be removed
02                              ;
03 INSERT    ST    R1,SaveR1    ; Save register we need
04           AND   R5,R5,#0     ; Set R5 to success code
05                             ; Initialization complete
06           LD    R1,NEG_LAST
07           ADD   R1,R1,R4     ; R1 = REAR MINUS x8005
08           BRnp  SKIP1        ; SKIP WRAP AROUND
09           LD    R4,FIRST     ; WRAP AROUND, R4=x8000
0A           BR    SKIP2
0B SKIP1     ADD   R4,R4,#1     ; NO WRAP AROUND, R4=R4+1
0C SKIP2     NOT   R1,R4
0D           ADD   R1,R1,#1     ; R1= NEG REAR
0E           ADD   R1,R1,R3     ; R1= FRONT-REAR
0F           BRz   FULL
10           STR   R0,R4,#0     ; DO THE INSERT
11           BR    DONE
12 FULL      LD    R1,NEG_FIRST
13           ADD   R1,R1,R4     ; R1 = REAR MINUS x8000
14           BRnp  SKIP3
15           LD    R4,LAST      ; UNDO WRAP AROUND, REAR=x8005
16           BR    SKIP4
17 SKIP3     ADD   R4,R4,#-1    ; NO WRAP AROUND, R4=R4-1
18 SKIP4     ADD   R5,R5,#1     ; R5=FAILURE
19           BR    DONE
1A                              ;
1B REMOVE    ST    R1,SaveR1    ; Save register we need
1C           AND   R5,R5,#0     ; Set R5 to success code
1D                             ; Initialization complete
1E           NOT   R1,R4
1F           ADD   R1,R1,#1     ; R1= NEG REAR
20           ADD   R1,R1,R3     ; R1= FRONT-REAR
21           BRz   EMPTY
22           LD    R1, NEG_LAST
23           ADD   R1,R1,R3     ; R1= FRONT MINUS x8005
24           BRnp  SKIP5
25           LD    R3, FIRST    ; R3=x8000
26           BR    SKIP6
27 SKIP5     ADD   R3,R3,#1     ; R3=R3+1
28 SKIP6     LDR   R0,R3,#0     ; DO THE REMOVE
29           BR    DONE
2A EMPTY     ADD   R5,R5.#1     ; R5=FAILURE
2B DONE      LD    R1,SaveR1    ; Restore register
2C           RET
2D FIRST     .FILL x8000
2E NEG_FIRST .FILL x8000
2F LAST      .FILL x8005
30 NEG_LAST  .FILL x7FFB
31 SaveR1    .BLKW 1
```

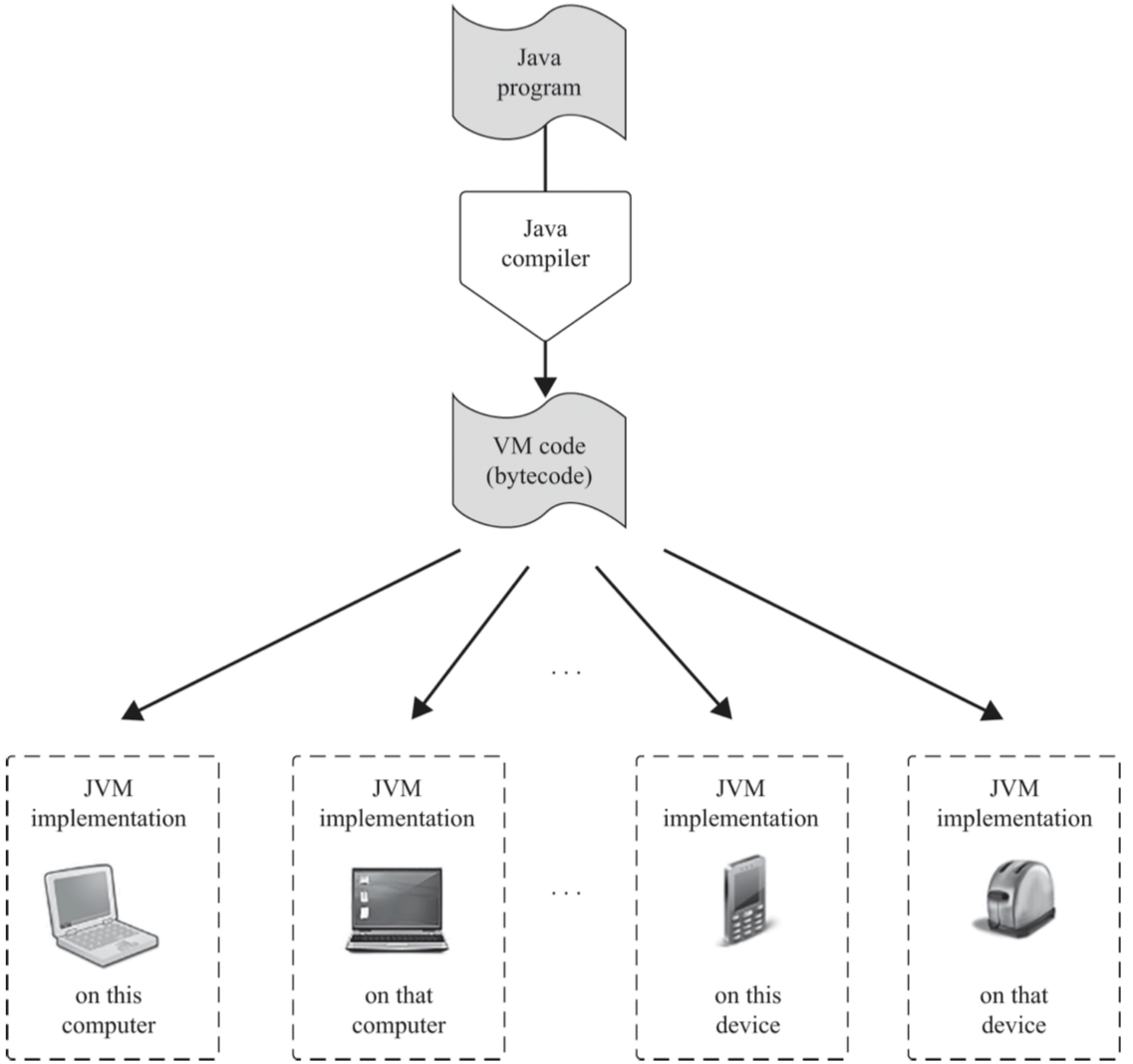
Stack Application: Java Virtual Machine

Virtual Machine: Another Intermediate Representation



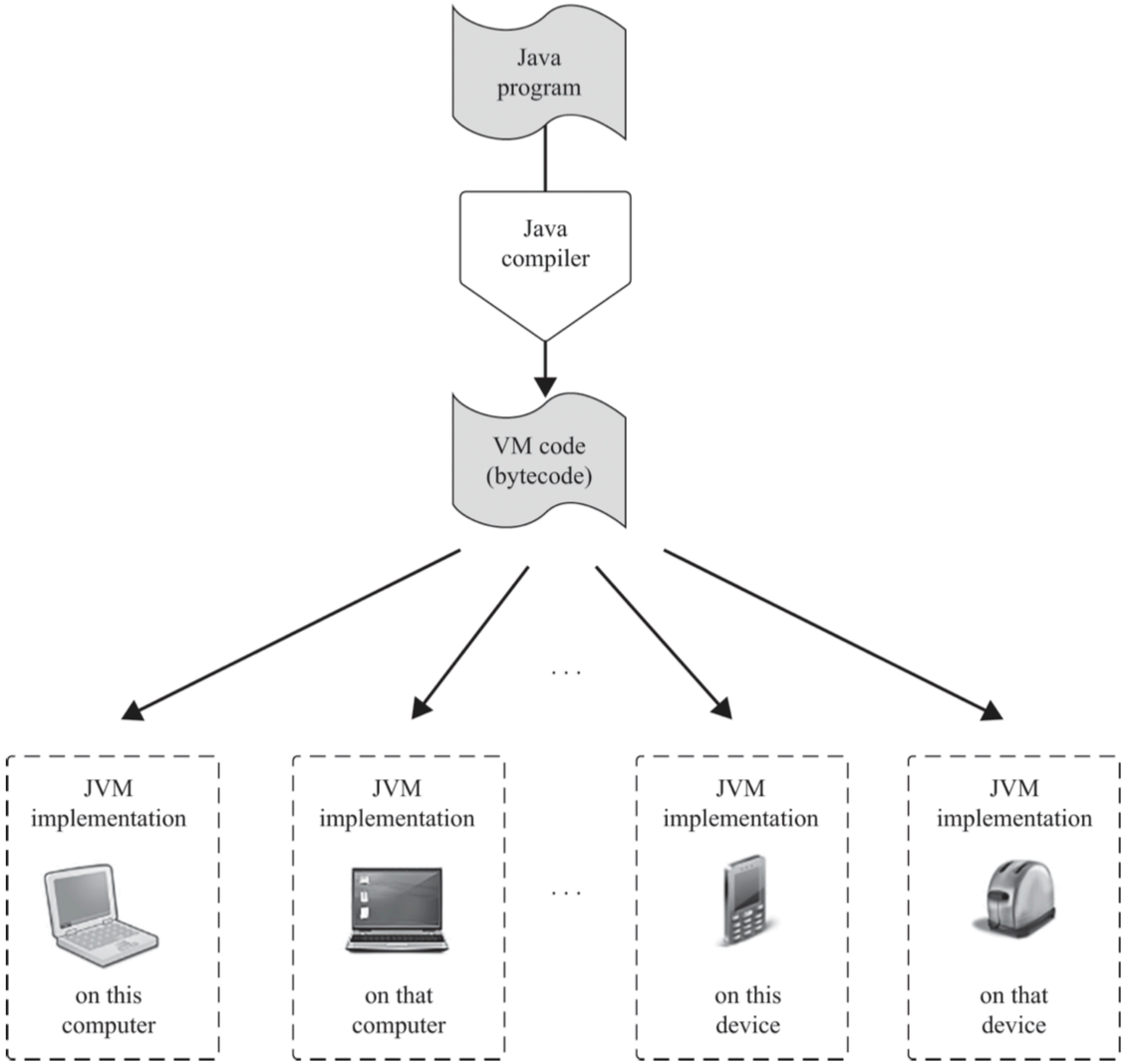
Virtual Machine: Java's Implementation

- High-level programs are compiled into intermediate VM code. The same VM code can be shipped to, and executed on, any hardware platform equipped with a suitable JVM implementation. These VM implementations are typically realized as client-side programs that translate the VM code into the machine languages of the target devices.

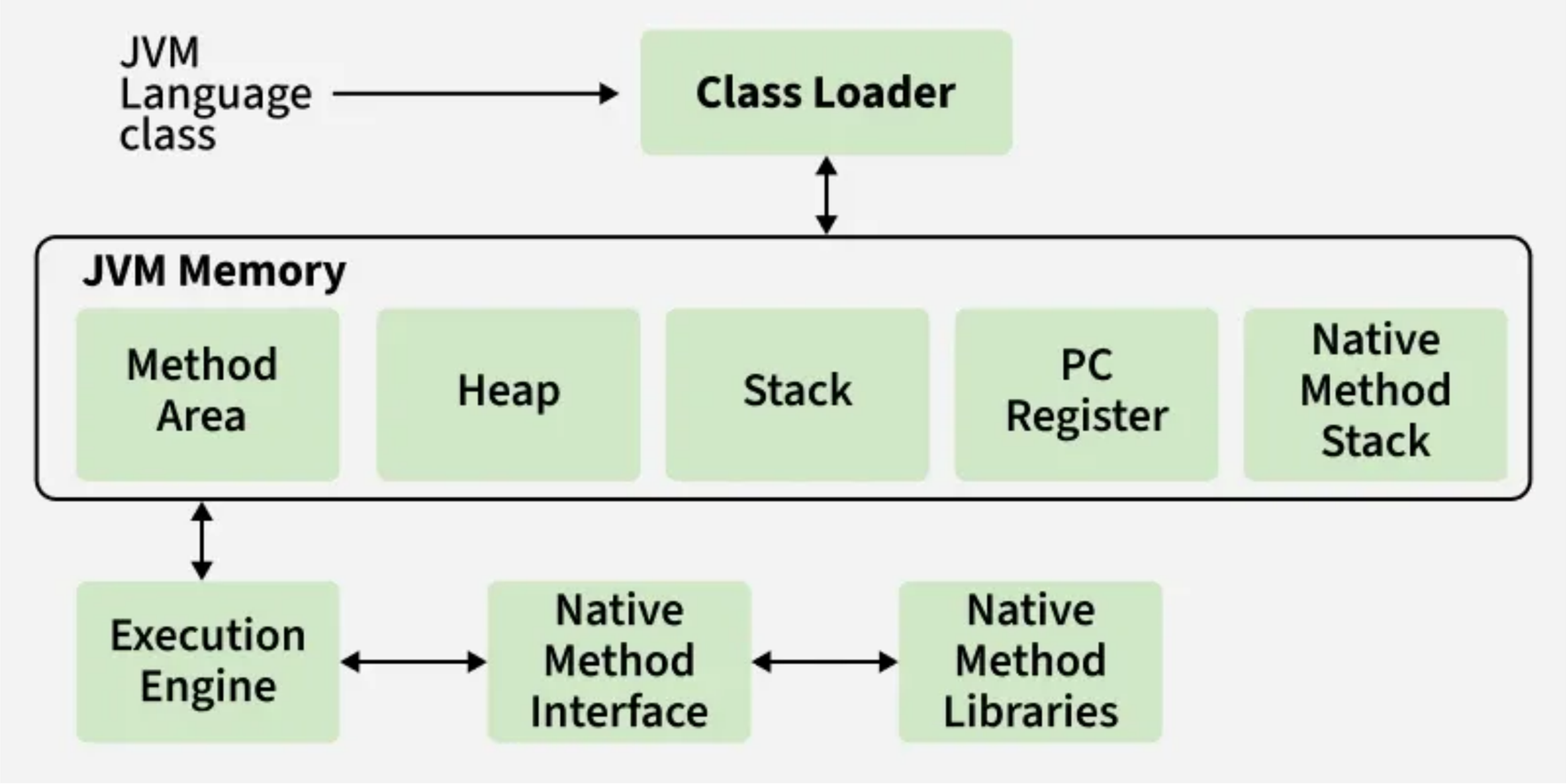


Virtual Machine: Java's Implementation

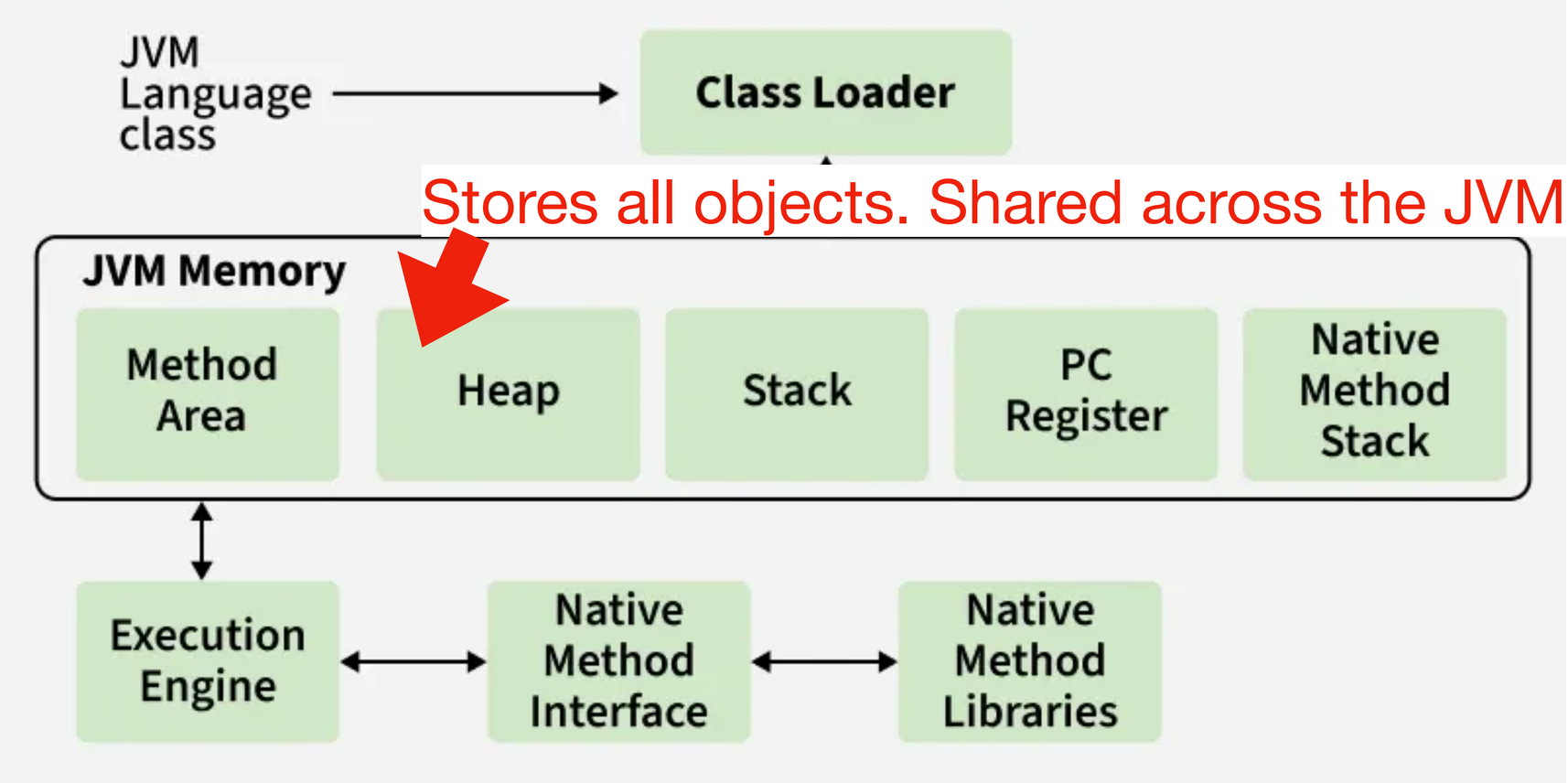
- High-level programs are compiled into intermediate VM code. The same VM code can be shipped to, and executed on, any hardware platform equipped with a suitable JVM implementation. These VM implementations are typically realized as client-side programs that translate the VM code into the machine languages of the target devices.



Application: Java Virtual Machine



Application: Java Virtual Machine



Application: Java Virtual Machine

Each thread has its own runtime stack; stores method calls, local variables in stack frames. Destroyed when the thread ends.

